DBMS EXPERIENTIAL LEARNING · LEVEL 3 PROJECT

SkillGraph

Personal Career Intelligence & Skill Development Engine

An evidence-driven decision system that tells a specific person what to learn next, why it matters for their target career, and how to prove it. Built on PostgreSQL with pgvector, MongoDB and Neo4j behind a Next.js application.

PostgreSQL 16 | pgvector · HNSW | MongoDB | Neo4j | Next.js 15 | 3 roles · RBAC | 50 automated tests

Team	Aditya Kalloli (PES1UG24AM015) · Abhay Bharadwaj (PES1UG24AM007)
Course	DBMS Experiential Learning, Level 3
Date	November 2026
Repository	github.com/AdityaK2204/skillgraph

Contents

1. Problem statement
2. Objectives
3. Scope
4. Functional requirements
5. ER diagram
6. Relational schema
7. Database choice and justification
8. Data dictionary
9. Implementation
10. Important queries
11. System architecture
12. Application screenshots
13. Testing results
14. Conclusion and future enhancements
15. Team contribution
Appendix A. Requirement compliance
Appendix B. Demonstration script (about 12 minutes)
Appendix C. Data dictionary (generated)

1. Problem statement

Students and early-career professionals can reach thousands of courses, job postings and skill lists, yet they cannot easily tell:

- which skills matter most for the career they want,
- which of those they are actually missing,
- which gaps to close first, and
- what evidence would make their profile credible.

Resume analysers answer "*how well do I match this job?*" They treat every skill written on a resume as fact, and they give no order of work. SkillGraph answers a different question: **"What should this specific person do next, why, and how can they prove it?"**

This is a database problem at heart. The answer depends on joining several kinds of data that have different shapes:

DATA	SHAPE
A person's claims and verified evidence	strongly related records that need integrity
Resumes and job postings from different sources	semi-structured documents
Skill prerequisites and role requirements	a graph
Meaning-level similarity between a profile and a posting	vectors

2. Objectives

1. Model career data in a normalised relational schema, with integrity enforced by keys, constraints and triggers rather than application code.
2. Keep **claims and evidence apart**. A skill's verified level can only be raised by mentor-reviewed evidence, and it is derived by a trigger.
3. Rank missing skills with an **explainable** priority built from seven factors: demand, relevance, gap, readiness, interest, evidence impact and ease. The ranking is computed inside PostgreSQL.
4. Generate a versioned, prerequisite-ordered **Learn → Build → Prove** roadmap in one transaction.
5. Store resumes and raw job postings as flexible documents (MongoDB) and analyse them with aggregation pipelines.
6. Represent the skill knowledge graph for multi-hop traversal (Neo4j), with a relational fallback.
7. Use vector embeddings (pgvector) for semantic job matching and interest-aware ranking.
8. Deliver a secure Next.js application with three roles and a clean API layer.

3. Scope

In scope

- Student self-registration and profiles.
- Target-role selection.
- Resume upload (PDF/TXT) with rule-based skill extraction.
- Skill claims, projects and evidence submission.
- Mentor review of evidence.
- Gap analysis and roadmap generation.
- Skill-graph exploration.
- Job search and hybrid job matching.
- Admin management of users, the skill catalogue, prerequisites, scoring weights, job ingestion, market signals and analytics.
- An audit trail.

Out of scope (future work)

- Live scraping of job portals (postings are ingested through the same pipeline from seed data and an admin form).
- LLM-based resume parsing.
- Course marketplaces and payments.
- Email notifications.
- OCR of scanned PDFs.

The demo dataset is fictional: 7 students, 1 mentor, 1 admin, 61 skills, 8 career roles, 35 job postings.

4. Functional requirements

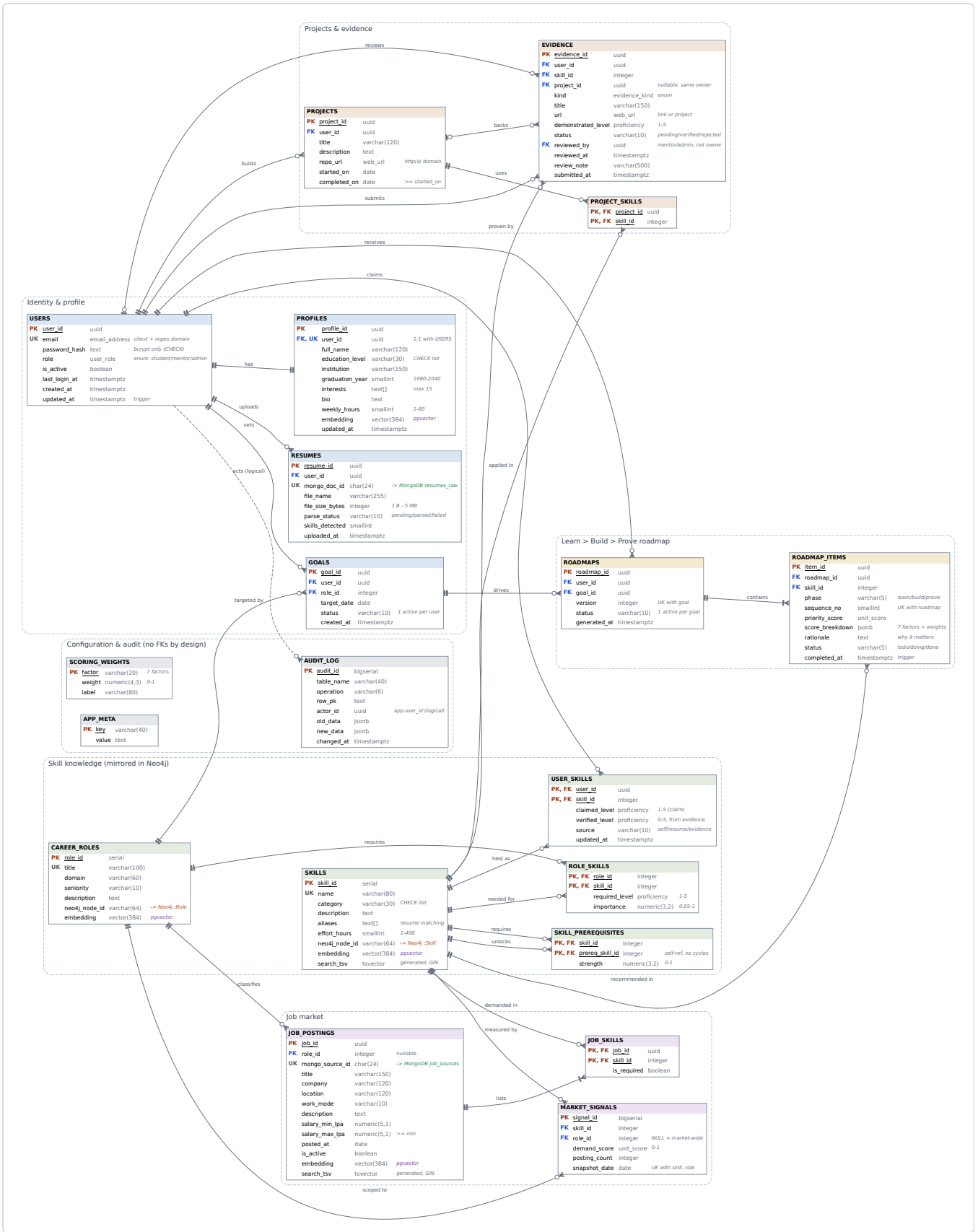
ID	ROLE	REQUIREMENT
FR-01	All	Register as a student, sign in and sign out. Passwords are hashed; sessions expire.
FR-02	All	Pages and APIs are restricted by role (student, mentor, admin).
FR-03	Student	Maintain a profile: education, institution, interests, bio, weekly learning hours.
FR-04	Student	Choose a target role (ranked by semantic fit). Only one goal is active at a time; changing it creates a new roadmap.
FR-05	Student	Upload a resume (PDF ≤ 5 MB or text) and see detected skills with mentions, sections and suggested levels.
FR-06	Student	Import selected resume skills as claims in a single transaction.
FR-07	Student	Add, update and remove skill claims. Verified skills cannot be removed.
FR-08	Student	Record projects and the skills they used.
FR-09	Student	Submit evidence (link or project) for a skill at a level, and withdraw it while it is pending.
FR-10	Student	See a gap analysis: readiness %, the status of each requirement, and ranked priorities with a per-factor breakdown.
FR-11	Student	Generate and track a Learn → Build → Prove roadmap with rationale, effort estimate and version history.
FR-12	Student	Explore the prerequisite graph for any role, find the shortest prerequisite path, and see related roles.
FR-13	Student	See job matches (structured + semantic) with missing required skills. Search jobs with full-text and filters.
FR-14	Mentor	Review pending evidence: verify (raises the verified level) or reject with a note.
FR-15	Admin	View analytics combining PostgreSQL aggregates and MongoDB aggregations.
FR-16	Admin	Manage users: change role, activate or deactivate.
FR-17	Admin	Manage the skill catalogue, prerequisites (cycles rejected) and scoring weights.
FR-18	Admin	Ingest job postings (raw → MongoDB → normalised PostgreSQL) and refresh market signals.
FR-19	Admin	Browse the audit log of role changes, reviews, catalogue edits and weight changes.

Non-functional requirements

- Data integrity is enforced by the database.
- No credentials in source code.
- Parameterised queries throughout.
- Usable at phone width, in light and dark mode.
- Graph features degrade gracefully when Neo4j is down.

5. ER diagram

The ER diagram uses crow's-foot notation. There are 20 entities, grouped by domain. Colour-coded columns point to the other stores: green to MongoDB, orange to Neo4j, purple to pgvector.



ER diagram

Key relationship types:

RELATIONSHIP	CARDINALITY	RESOLVED BY
User has Profile	1 : 1	`profiles.user_id` UNIQUE

RELATIONSHIP	CARDINALITY	RESOLVED BY
User holds Skill	M : N	`user_skills` (claimed + verified level)
Skill requires Skill	recursive M : N	`skill_prerequisites` (acyclic, enforced by trigger)
Role requires Skill	M : N	`role_skills`
Project uses Skill · Posting lists Skill	M : N	`project_skills` · `job_skills`
User targets Role	M : N over time	`goals` (one active per user, partial unique index)
Evidence proves Skill / backed by Project / reviewed by User	N:1 / N:0..1 / N:0..1	FKs on `evidence`
Goal drives Roadmap	1 : N (versions)	`roadmaps(goal_id, version)` UNIQUE
Roadmap contains Items	1 : 1..N, weak entity	`roadmap_items`, cascade delete

6. Relational schema

Notation: underlined = primary key (shown here as **bold**), *italic* = foreign key.

- users(**user_id**, email UK, password_hash, role, is_active, last_login_at, created_at, updated_at)
- profiles(**profile_id**, *user_id* UK → users, full_name, education_level, institution, graduation_year, interests[], bio, weekly_hours, embedding, updated_at)
- resumes(**resume_id**, *user_id* → users, mongo_doc_id UK, file_name, file_size_bytes, parse_status, skills_detected, uploaded_at)
- career_roles(**role_id**, title UK, domain, seniority, description, neo4j_node_id, embedding, created_at)
- goals(**goal_id**, *user_id* → users, *role_id* → career_roles, target_date, status, created_at)
- skills(**skill_id**, name UK, category, description, aliases[], effort_hours, neo4j_node_id, embedding, search_tsv, created_at)
- skill_prerequisites(***skill_id*** → skills, ***prereq_skill_id*** → skills, strength)
- role_skills(***role_id*** → career_roles, ***skill_id*** → skills, required_level, importance)
- user_skills(***user_id*** → users, ***skill_id*** → skills, claimed_level, verified_level, source, updated_at)
- projects(**project_id**, *user_id* → users, title, description, repo_url, started_on, completed_on, created_at)
- project_skills(***project_id*** → projects, ***skill_id*** → skills)
- evidence(**evidence_id**, *user_id* → users, *skill_id* → skills, *project_id* → projects, kind, title, url, demonstrated_level, status, *reviewed_by* → users, reviewed_at, review_note, submitted_at)
- job_postings(**job_id**, *role_id* → career_roles, mongo_source_id UK, title, company, location, work_mode, description, salary_min_lpa, salary_max_lpa, posted_at, is_active, embedding, search_tsv, created_at)
- job_skills(***job_id*** → job_postings, ***skill_id*** → skills, is_required)
- market_signals(**signal_id**, *skill_id* → skills, *role_id* → career_roles, demand_score, posting_count, snapshot_date) · UK(skill_id, role_id, snapshot_date) NULLS NOT DISTINCT
- roadmaps(**roadmap_id**, *user_id* → users, *goal_id* → goals, version, status, generated_at) · UK(goal_id, version)
- roadmap_items(**item_id**, *roadmap_id* → roadmaps, *skill_id* → skills, phase, sequence_no, priority_score, score_breakdown, rationale, status, completed_at) · UK(roadmap_id, sequence_no), UK(roadmap_id, skill_id)
- scoring_weights(**factor**, weight, label)
- audit_log(**audit_id**, table_name, operation, row_pk, actor_id, old_data, new_data, changed_at)
- app_meta(**key**, value)

Normalisation. Every table has a single-purpose key, and every non-key attribute depends on the whole key and nothing else. Each M:N relationship is its own table with a composite key and relationship attributes (levels, importance, is_required). The schema is in 3NF, and BCNF for all user-maintained tables.

Four controlled redundancies are deliberate:

REDUNDANCY	WHY IT EXISTS	HOW IT STAYS CONSISTENT
`user_skills.verified_level`	Fast reads	Derived from verified evidence by trigger, never written by the app
`search_tsv`	Full-text search	`GENERATED ALWAYS` column
`market_signals`	Daily demand snapshots	Rebuilt by a procedure

REDUNDANCY	WHY IT EXISTS	HOW IT STAYS CONSISTENT
`mv_skill_demand`	Analytics	Materialized view

7. Database choice and justification

Each database is chosen for the shape of the data it holds and the operations the system performs on it.

DATA	STORE	WHY THIS STORE
Users, profiles, goals, claims, evidence, projects, roles, requirements, normalised jobs, roadmaps	**PostgreSQL**	Strongly related records that must stay consistent: foreign keys, CHECK and domain constraints, unique and partial-unique rules, multi-table transactions (resume import, roadmap generation, evidence review), triggers for derived values and audit, and SQL joins and aggregation for ranking.
Embeddings of profiles, skills, roles, postings	**pgvector** (inside PostgreSQL)	Semantic similarity must be combined with relational filters and joins in one query (e.g. `match_jobs` mixes skill overlap with cosine similarity). Keeping vectors next to the rows avoids syncing a separate vector store. HNSW gives approximate nearest-neighbour search.
Parsed resumes, raw job-source records, activity events	**MongoDB**	Structure varies by document. Resumes have different sections and entries, and each job portal names fields differently. The raw form is kept as delivered (with light `$jsonSchema` validation) for re-parsing and provenance. Aggregation pipelines answer questions like "which terms in resumes are missing from our catalogue". Activity events expire with a TTL index.
Skill prerequisites, role requirements, related skills, evidence paths	**Neo4j**	The core questions are multi-hop traversals: everything a role transitively requires, the shortest prerequisite chain between two skills, roles sharing a foundation, how a user's evidence connects to a skill. Cypher expresses these directly. PostgreSQL stays the source of truth; Neo4j is a read-optimised projection rebuilt by `graph:sync`, and every graph read has a recursive-CTE fallback.

8. Data dictionary

The complete, generated data dictionary is in `docs/DATA_DICTIONARY.md`: 20 tables, 205 columns, 51 CHECK constraints and 26 foreign keys. The tables below summarise the reusable types and the most important columns.

Domains and enums

TYPE	DEFINITION	USED BY
`email_address`	`citext` + regex CHECK	users.email (case-insensitive unique)
`proficiency`	`smallint` CHECK 0–5	claimed / verified / required / demonstrated levels
`unit_score`	`numeric(5,4)` CHECK 0–1	demand_score, priority_score
`web_url`	`varchar(300)` CHECK `^https?://`	repo_url, evidence.url
`user_role`	ENUM student, mentor, admin	users.role
`evidence_kind`	ENUM certificate, repository, project, assessment, publication	evidence.kind

Selected columns

TABLE.COLUMN	TYPE	MEANING / RULE
users.password_hash	text	CHECK `^\$2[aby]\$\d{2}\$.{53}$`: only bcrypt hashes are accepted
user_skills.claimed_level	proficiency ≥ 1	What the user says (self, resume or evidence)
user_skills.verified_level	proficiency	Max demonstrated level of *verified* evidence; maintained by trigger
evidence.status	varchar	pending / verified / rejected; CHECK `(status='pending') = (reviewed_at IS NULL)`
evidence.url / project_id	web_url / uuid	CHECK `url IS NOT NULL OR project_id IS NOT NULL`
skills.effort_hours	smallint 1–400	Typical hours to working level (drives the *ease* factor)
role_skills.importance	numeric 0.05–1	Weight of the requirement (drives *relevance* and readiness)
roadmap_items.score_breakdown	jsonb (object)	All seven factor values, levels, missing prerequisites and the weights used
market_signals.demand_score	unit_score	Weighted share of active postings asking for the skill
*.embedding	vector(384)	Unit-length sentence embedding

9. Implementation

9.1 PostgreSQL

All database code is in `db/postgres/`, about 1,400 lines of SQL.

DDL (01_schema.sql).

- Objects: 3 extensions (`vector`, `citext`, `pg_trgm`), 4 domains, 2 enums, 20 tables.
- Constraints: 51 CHECK constraints, 26 foreign keys with deliberate actions:
- `CASCADE` for owned data;
- `RESTRICT` for catalogue rows other data depends on;
- `SET NULL` for optional links.
- Indexes: 53 in total (including the ones keys create automatically). These include FK/join B-trees; partial indexes (`ix_evidence_pending`, plus partial unique `ux_goals_one_active` and `ux_roadmaps_one_active`); GIN full-text indexes on two generated `tsvector` columns; a GIN trigram index; and four HNSW indexes with `vector_cosine_ops`.

CREATE / ALTER / DROP. `demo/01_ddl_alter_drop.sql` creates a table and adds, alters, renames and drops columns and constraints. It shows a CHECK violation being caught, creates and drops an index and a view, and alters a core table. It runs inside a transaction and rolls back.

Views.

VIEW	PURPOSE
`v_user_skills`	Claims vs. evidence with effective level
`v_role_requirements`	Role requirements, readable
`v_active_goals`	Each user's active goal
`v_goal_gap`	Per-requirement gap and status (proven / met / partial / missing)
`v_evidence_queue`	Mentor review queue
`v_student_overview`	Admin view of every student
`mv_skill_demand` (materialized)	Demand vs. learner supply, with a unique index for concurrent refresh

Functions and procedure.

ROUTINE	KIND	WHAT IT DOES
`effective_level(claimed, verified)`	IMMUTABLE SQL	`verified + 0.5 × (claimed − verified)`: unproven claims count half
`compute_skill_priorities(user, role)`	SQL, recursive CTE	Candidates = the role's requirements plus all transitive prerequisites. For each real gap it computes 7 factors in [0,1] and a weighted priority (weights from `scoring_weights`, normalised by their sum).
`generate_roadmap(user, max_items)`	PL/pgSQL	Takes an advisory lock, archives the old version, inserts version + 1 and its items, ordered by prerequisite depth then priority. Assigns a phase (learn / build / prove) and a generated rationale. Raises `P0002` without a goal.
`match_jobs(user, limit)`	SQL	`0.6 × weighted skill overlap + 0.4 × cosine similarity`; returns matched and missing required skills
`goal_readiness(user)`	SQL	Importance-weighted readiness %

ROUTINE	KIND	WHAT IT DOES
`refresh_market_signals()`	PROCEDURE, SECURITY DEFINER	Rebuilds today's per-role and market-wide demand (recency-weighted) and refreshes the materialized view

The seven factors, each in [0,1]:

FACTOR	DEFAULT WEIGHT	DEFINITION
demand	0.20	Share of postings for the role that ask for the skill
relevance	0.20	Role importance; for prerequisites, inherited × strength × 0.8
gap	0.20	(required − effective) / required
readiness	0.10	Share of prerequisites at level 2 or above
interest	0.10	Cosine similarity between the profile and the skill embeddings
evidence	0.10	1.0 when claimed but unproven; 0.6 when unclaimed; 0 when already proven
ease	0.10	1 − ln(effort_hours) / ln(400)

Triggers. There are 14 triggers on 8 trigger functions.

TRIGGER	RULE IT ENFORCES
`evidence_guard`	A backing project must belong to the same student. Only a mentor or admin may review, never their own evidence. Sets or clears review timestamps.
`evidence_sync`	Recomputes `verified_level` (and lifts the claim to it) on insert, update or delete of evidence
`prereq_no_cycle`	Recursive check: rejects any edge that would make the prerequisite graph cyclic
`goal_single_active`	A new active goal retires the previous one
`item_status`, `roadmap_rollup`	Stamp `completed_at`; mark the roadmap completed when every item is done
`audit_*` (5 tables)	Append-only JSONB before/after images. Actor comes from `SET LOCAL app.user_id`; secrets and vectors are stripped.
`*_updated_at`	Timestamp maintenance

Transactions. Every multi-statement write in the API goes through `tx()`, which runs `BEGIN … COMMIT/ROLLBACK` and records the actor. This covers:

- registration (user + profile);
- goal change + first roadmap;
- resume skill import (all-or-nothing; tested by injecting an invalid skill);
- project + skills;
- evidence + claim;
- mentor review (with triggers firing);
- admin role changes, which include a check that at least one active admin remains.

`demo/03_transactions_dml.sql` demonstrates INSERT, UPDATE and DELETE, SAVEPOINT recovery from a trigger rejection, a domain CHECK and an FK RESTRICT, `ON DELETE SET NULL` interacting with a CHECK, and a final ROLLBACK.

Least privilege. The app connects as `skillgraph_app`, which has DML only: no CREATE or DROP, and no UPDATE or DELETE on `audit_log`. The procedure that refreshes the materialized view runs as `SECURITY DEFINER`.

9.2 MongoDB

Collections, validators and indexes are defined once in `db/mongo/schema.json`. Docker applies them through `init.js`; `npm run db:init` applies them outside Docker.

COLLECTION	DOCUMENT	VALIDATION	INDEXES
`resumes_raw`	`{userId, fileName, mimeType, text, sections{heading → lines}, extracted{skills[{skillId, name, matchedTerm, mentions, sections, suggestedLevel}], unknownTerms, education, projects, links}, parser{name, version}, createdAt}`	`$jsonSchema` (required fields, int levels 1–5, mime enum)	`{userId, createdAt}`, multikey `extracted.skills.skillId`, text index on `text`
`job_sources`	`{source, externalId, payload{…as delivered}, normalized{title, company, location, skills}, status, pgJobId, unresolvedSkills, error, ingestedAt}`	`$jsonSchema` (status enum)	unique `{source, externalId}`, `{status, ingestedAt}`
`activity_log`	`{userId, type, meta, at, day}`	`$jsonSchema` (event-type enum, day pattern)	`{userId, at}`, `{day}`, **TTL** on `at` (180 days)

Cross-store consistency.

- PostgreSQL keeps `resumes.mongo_doc_id` and `job_postings.mongo_source_id`, both as UNIQUE `char(24)` with a hex CHECK.
- The resume upload writes MongoDB first, then PostgreSQL. If the PostgreSQL insert fails, the MongoDB document is deleted (a compensating action).
- Job ingestion records `received → normalized | rejected` in MongoDB, so failed records keep their error message.

Ingestion example. Two seed "portals" deliver differently shaped records:

SOURCE	SHAPE
portal-a	`jobTitle`, `org`, `skillsCsv`
portal-b	nested `position{}` / `employer{}`, `tags[]`, a salary string
manual	the admin form

Adapters normalise all three. Skill names and aliases are resolved in SQL, the role is inferred from requirement overlap if not given, and the posting is embedded and written to PostgreSQL in one transaction.

Aggregations (`db/mongo/aggregations.js`; also used by the admin analytics API):

- the most-detected skills across resumes (`$unwind → $group → $sort`);
- **catalogue gaps**: resume terms with no matching skill;
- ingestion by source and status;
- requested skills per source (two-stage `$group`);
- activity per day with distinct users;
- `$text` search inside resumes.

9.3 Neo4j

Model.

- `(:Skill)-[:PREREQ_OF {strength}]->(:Skill)`
- `(:Role)-[:REQUIRES {level, importance}]->(:Skill)`
- `(:Skill)-[:RELATED_TO {weight}]->(:Skill)`, from co-occurrence in ≥ 2 postings
- `(:User)-[:HAS_SKILL {claimed, verified}]->(:Skill)`
- `(:User)-[:TARGETS]->(:Role)`
- `(:User)-[:BUILT]->(:Project)-[:USES]->(:Skill)`
- `(:User)-[:SUBMITTED]->(:Evidence)-[:PROVES]->(:Skill)`, with `(:Evidence)-[:FROM_PROJECT]->(:Project)`

Constraints. Uniqueness on each node's PostgreSQL key, plus lookup indexes.

Sync. `npm run graph:sync` projects the whole catalogue and every user. Writes in the app call `syncUserQuietly()`, and catalogue edits re-project the graph. PostgreSQL stores `neo4j_node_id` for traceability.

Queries (`db/neo4j/02_queries.cypher`, plus `lib/graph.ts`):

- the learning path to a role (variable-length `PREREQ_OF*0..6`);
- `shortestPath` between two skills;
- gaps that also block other requirements;
- evidence paths;
- keystone skills;
- related skills;
- pivot roles.

Fallback. If Neo4j is unreachable (a health check cached for 30 s), the same answers come from PostgreSQL recursive CTEs. The UI states which engine answered.

9.4 Vector database (pgvector)

QUESTION	ANSWER
What is embedded	**Skills:** name + description + aliases. **Roles:** title + description + ordered requirements. **Postings:** title + description + skills. **Profiles:** target role + interests + bio + top skills, re-embedded whenever these change.
Model	`sentence-transformers/all-MiniLM-L6-v2` (ONNX via Transformers.js, run locally: no API key, no data leaves the machine). Mean pooling, L2-normalised, **384 dimensions**. Offline fallback: deterministic feature hashing of unigrams, bigrams and character trigrams (384-d). `app_meta.embedding_model` records which model produced the stored vectors; `npm run embed` re-embeds everything after a switch.
Storage	`vector(384)` columns in `profiles`, `skills`, `career_roles`, `job_postings`, each with an **HNSW** index (`vector_cosine_ops`)
Metric	Cosine distance `<=>`; similarity = 1 − distance
Retrieval	(1) k-NN `ORDER BY embedding <=> $q LIMIT k` for similar postings (the HNSW index is verified with EXPLAIN in the tests). (2) Hybrid scoring in `match_jobs`. (3) The interest factor in `compute_skill_priorities`. (4) Ranking target roles by profile fit.

10. Important queries

All queries are in `db/postgres/demo/02_important_queries.sql`; outputs below are from the seeded database.

Q3: GROUP BY + HAVING + aggregate FILTER

```
SELECT s.name, count(*) AS postings, count(*) FILTER (WHERE js.is_required) AS required_in,
       round(100.0 * count(*) / (SELECT count(*) FROM job_postings WHERE is_active), 1) AS
pct_of_postings
FROM job_skills js JOIN job_postings jp ON jp.job_id = js.job_id AND jp.is_active
JOIN skills s ON s.skill_id = js.skill_id
GROUP BY s.skill_id, s.name HAVING count(*) >= 5 ORDER BY postings DESC;
```

Top rows: Python, 19 postings (54.3%); SQL, 17 (48.6%); Docker, 13 (37.1%).

Q4: correlated `NOT EXISTS` subquery. Skills claimed at level 3 or above with no verified evidence; this is the claims-vs-evidence gap.

```
SELECT u.email, s.name, us.claimed_level FROM user_skills us JOIN users u USING (user_id) JOIN skills s
USING (skill_id)
WHERE us.claimed_level >= 3 AND NOT EXISTS (SELECT 1 FROM evidence e WHERE e.user_id = us.user_id
      AND e.skill_id = us.skill_id AND e.status = 'verified');
```

Q6: recursive CTE. Every transitive prerequisite of Machine Learning, with its depth and path. For example: Probability, depth 2, via Machine Learning ← Statistics.

Q7: window function. Students ranked by readiness within each target role:

```
SELECT ag.role_title, p.full_name, goal_readiness(ag.user_id) AS readiness_pct,
       rank() OVER (PARTITION BY ag.role_id ORDER BY goal_readiness(ag.user_id) DESC) AS rank_in_role
FROM v_active_goals ag JOIN profiles p USING (user_id);
```

Q8: the 7-factor priority function for the Data Analyst student:

SKILL	REQ	EFF	DEMAND	RELEVANCE	GAP	READINESS	INTEREST	EVIDENCE	EASE	PRIORITY
SQL	4	3.0	1.00	1.00	0.25	1.00	0.14	0.60	0.32	**0.656**
Data Visualization	4	1.0	0.64	0.90	0.75	0.00	0.23	1.00	0.43	**0.624**
Data Cleaning	3	0	0.36	0.80	1.00	0.00	0.34	0.60	0.46	**0.573**
Power BI	3	0	0.32	0.60	1.00	0.50	0.17	0.60	0.43	**0.554**

Q9: full-text search. `WHERE search_tsv @@ websearch_to_tsquery('english', 'python dashboards') ORDER BY ts_rank(...)`. The API also returns `ts_headline` snippets.

Q10: vector k-NN. `ORDER BY jp.embedding <=> p.embedding LIMIT 5` over the HNSW index.

Q11: hybrid matching (`match_jobs`):

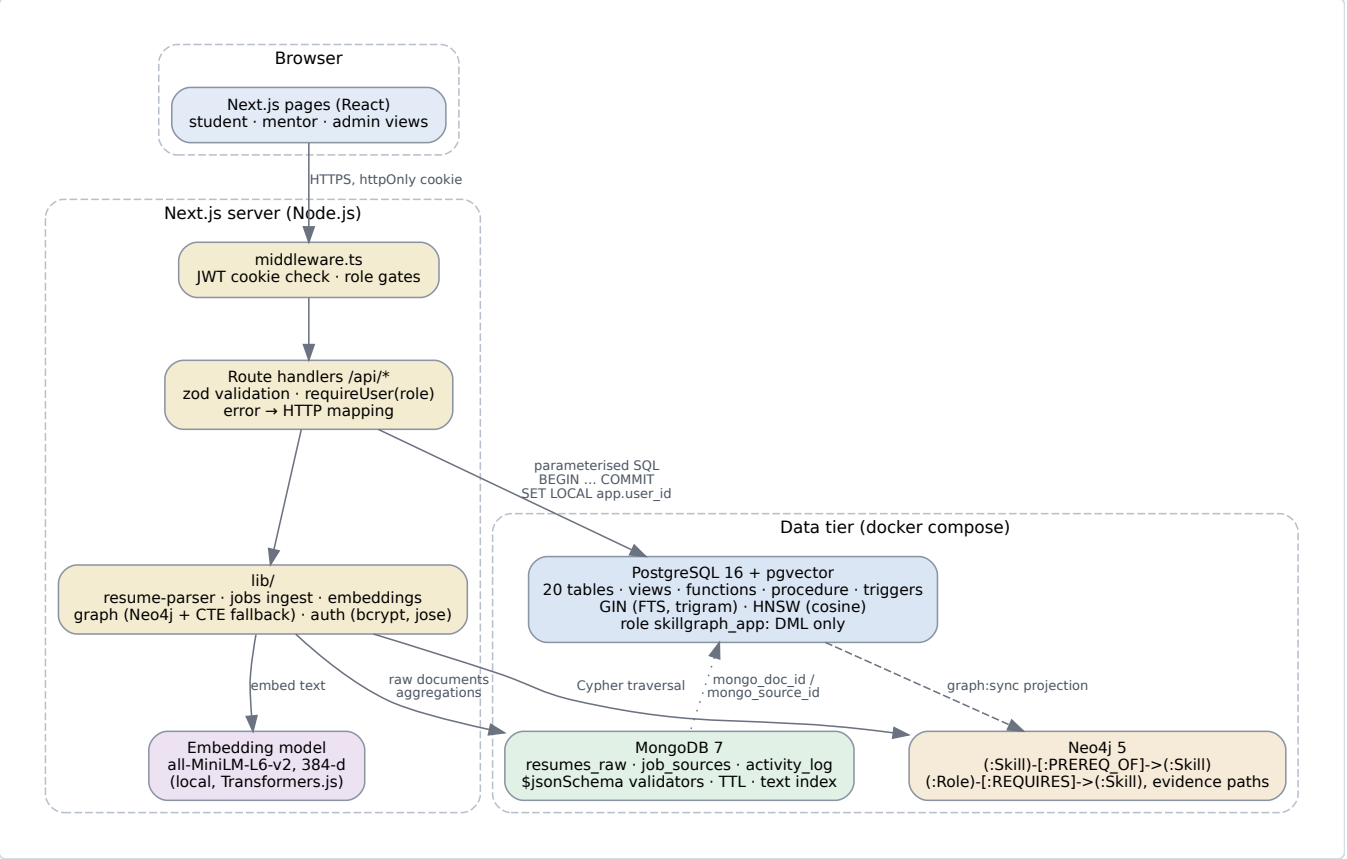
POSTING	STRUCTURED	SEMANTIC	MATCH	MISSING REQUIRED
Supply Chain Data Analyst	0.593	0.400	**0.515**	Data Visualization, Python
Junior Data Analyst	0.567	0.436	**0.514**	Data Visualization, Statistics
Data Analyst, Healthcare Ops	0.533	0.402	**0.481**	Communication, Data Cleaning

Q12: trigram "did you mean". `similarity(name, 'Postgress')` returns PostgreSQL with a score of 0.62.

Q14: EXPLAIN. The mentor queue uses `Index Scan using ix_evidence_pending`.

Q15: audit trail. Joins `audit_log` to users to show the actor, with old and new status.

11. System architecture



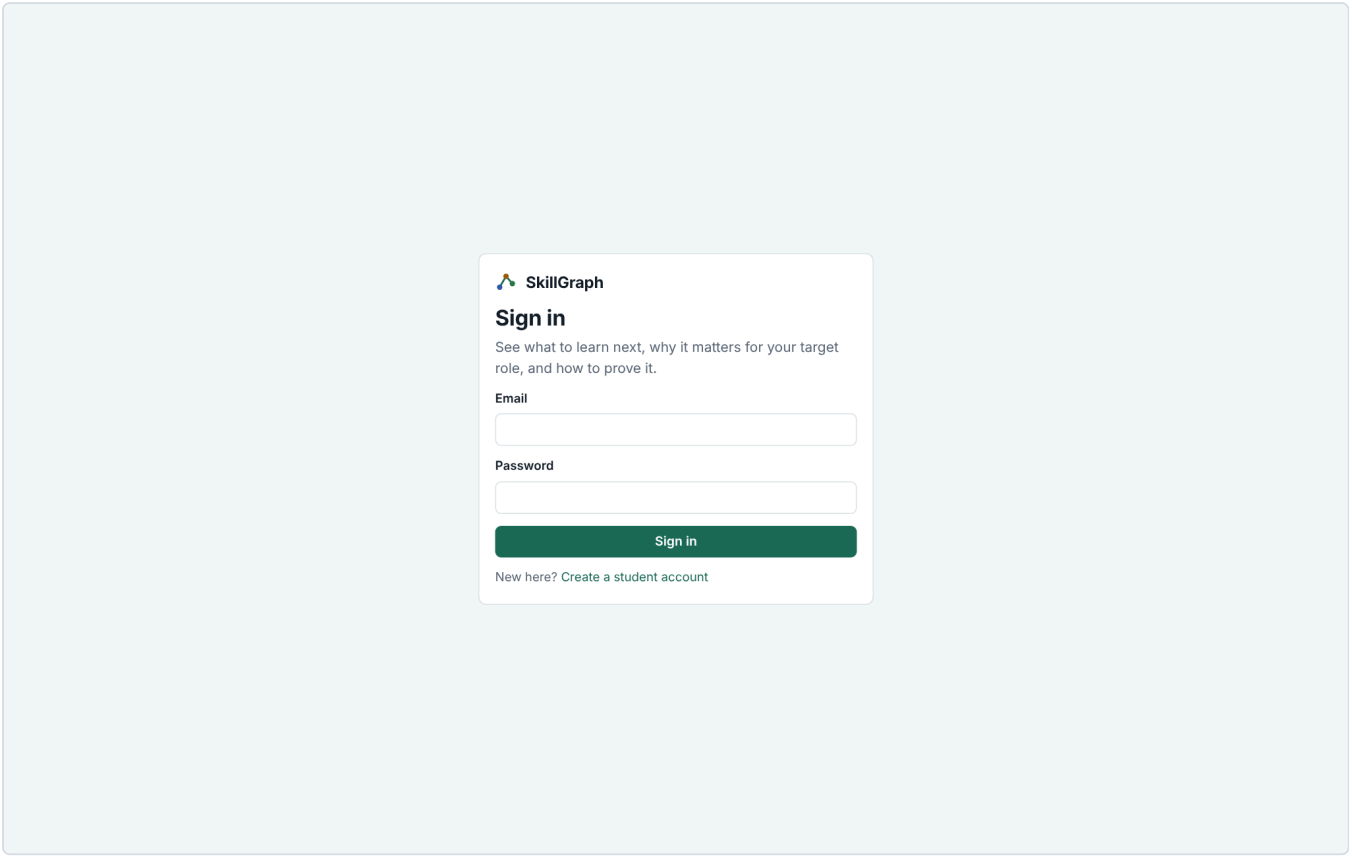
System architecture

- **Browser.** React pages under `app/(app)/…` call only `/api/*` and never touch a database.
- **Next.js server.**
- `middleware.ts` verifies the JWT cookie and applies role gates.
- Each route handler validates input with zod, re-checks the user and role against PostgreSQL (`requireUser`), runs parameterised SQL (inside `tx()` for multi-statement writes), and maps database errors to HTTP codes. For example, `23514` (CHECK or trigger rule) becomes 422, `23505` becomes 409, `42501` becomes 403 and `P0002` becomes 404.
- **Configuration.** Read from environment variables and validated at start-up (`lib/env.ts`).
- **Data tier.** Run by `docker compose`, with ports bound to `127.0.0.1` and a least-privilege user in both PostgreSQL and MongoDB.

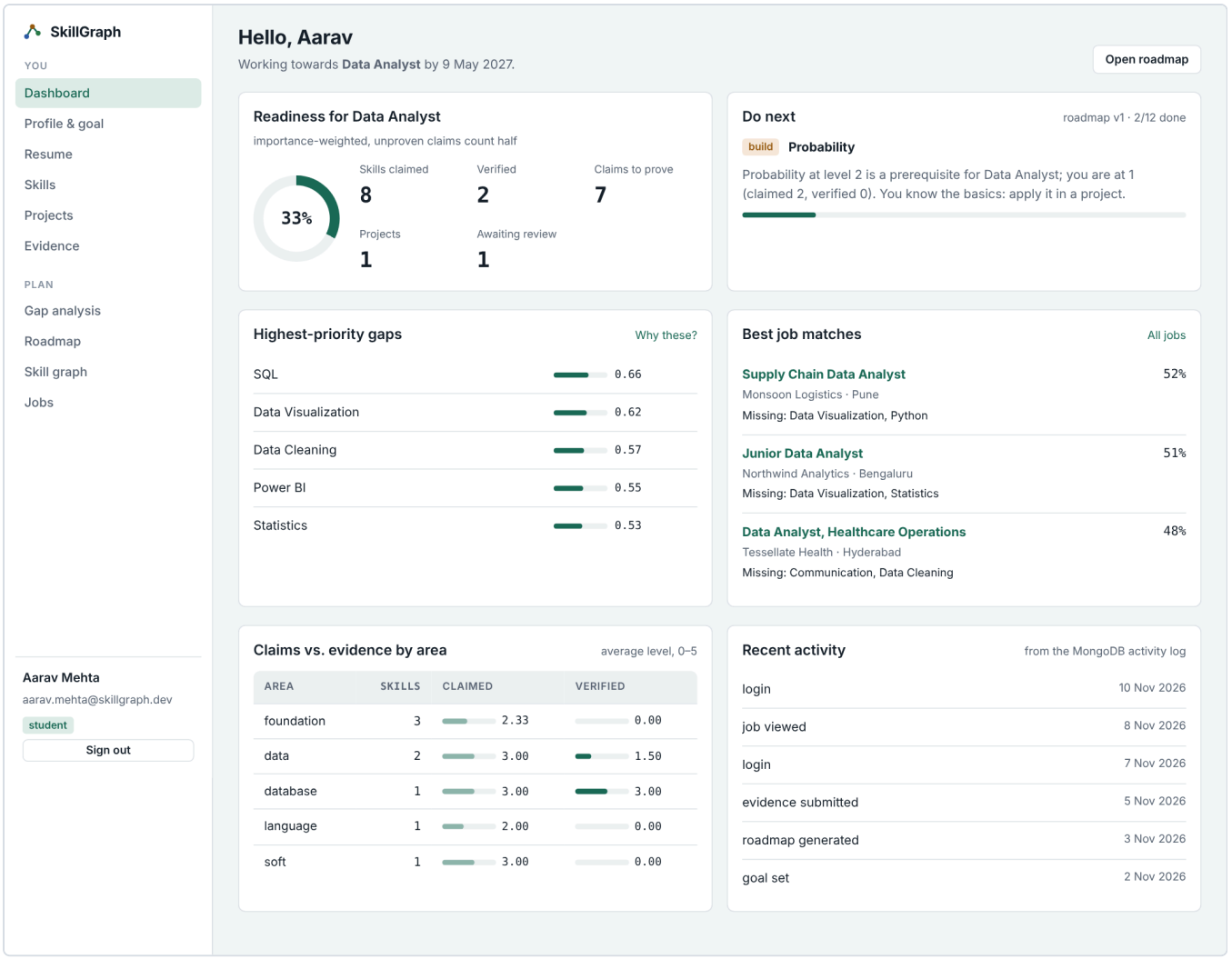
Security summary.

- bcrypt (cost 12) passwords, with a database CHECK that only hashes are stored;
- an HS256 JWT in an httpOnly, SameSite cookie;
- a role re-check on every request (deactivation takes effect immediately; tested);
- a timing-safe login with a failure throttle;
- an upload type and size check, including the PDF magic number;
- full-text snippets sanitised to `<mark>` only;
- security headers;
- `npm audit` reports 0 vulnerabilities.

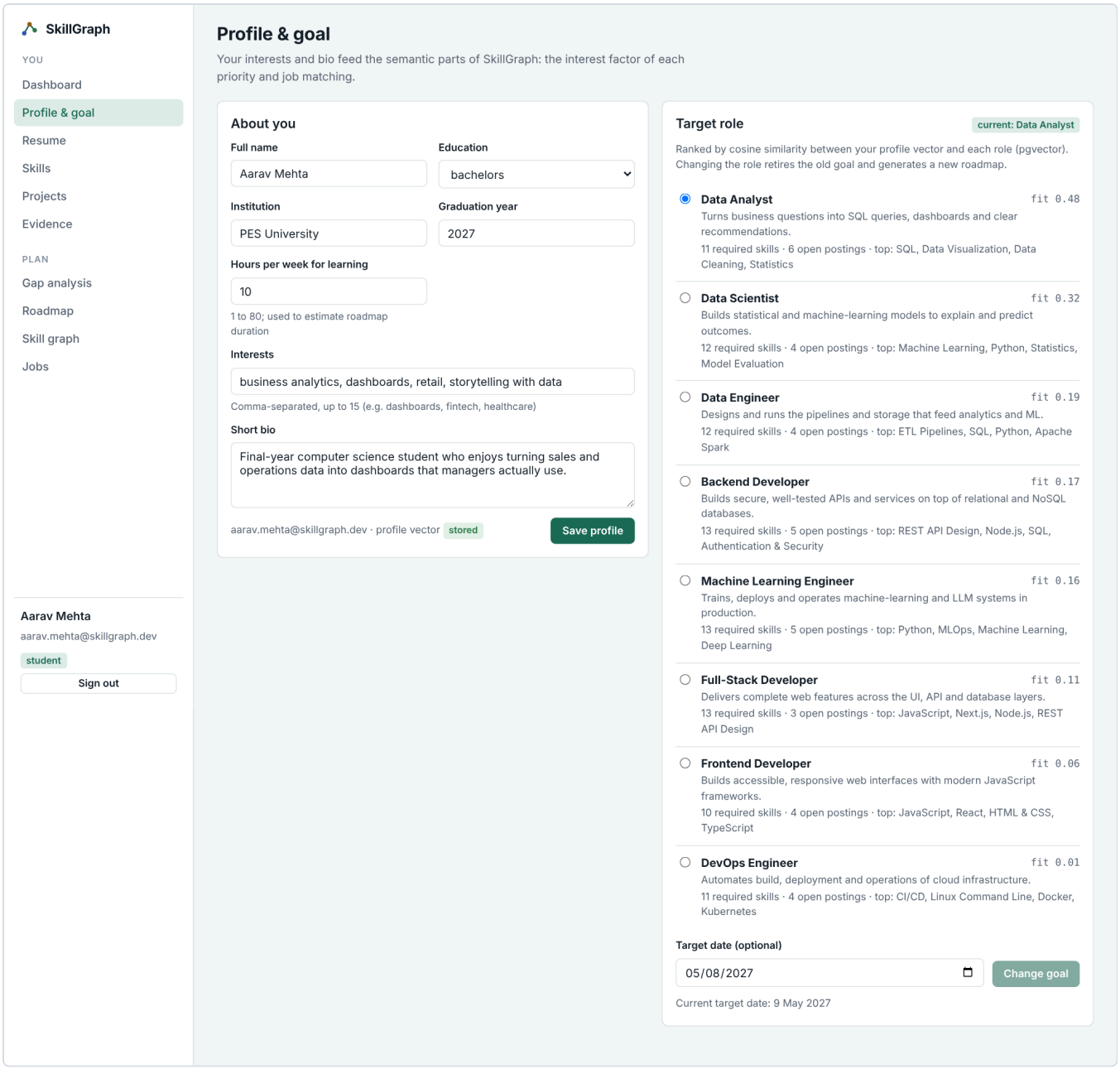
12. Application screenshots



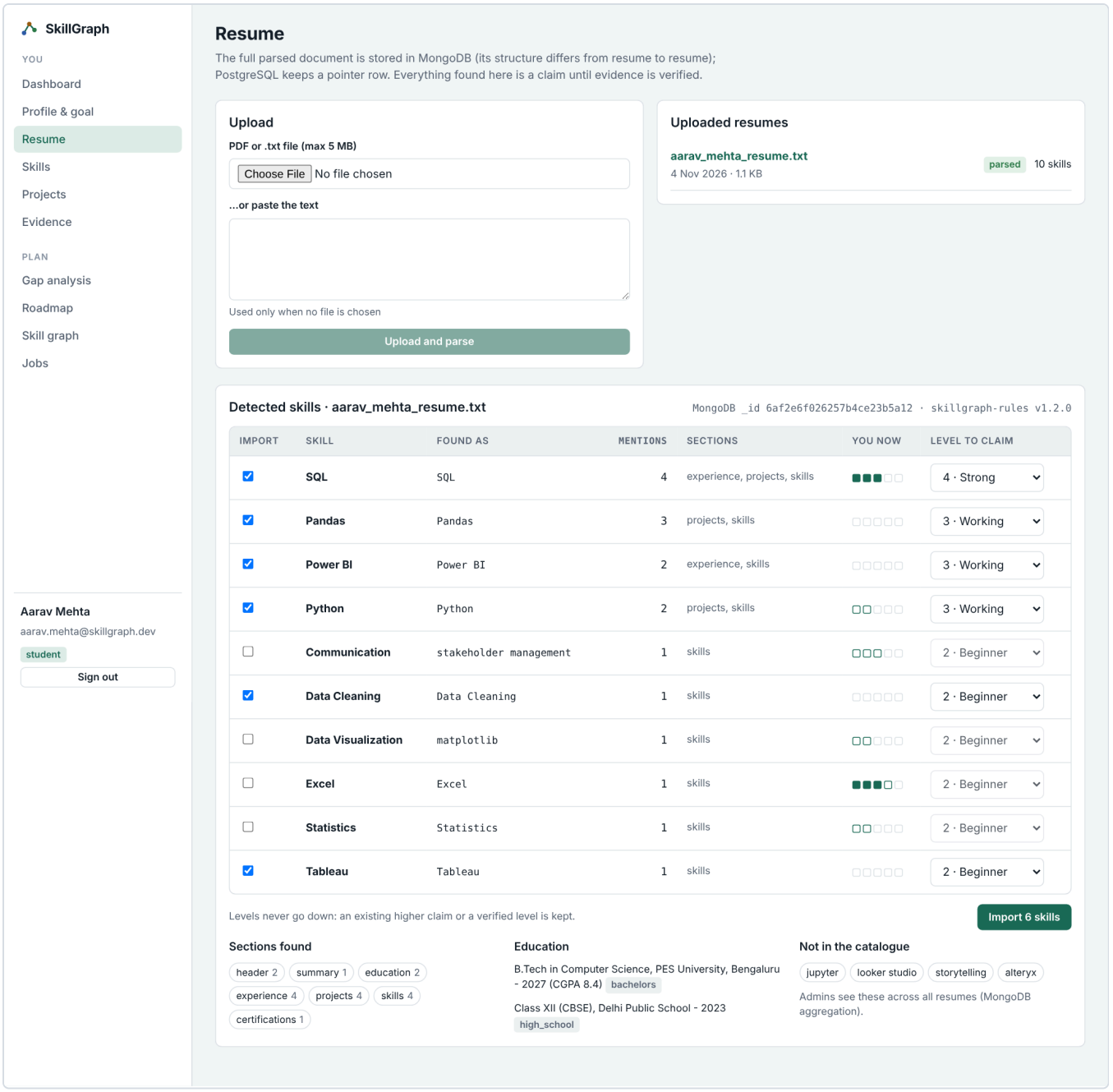
Sign in



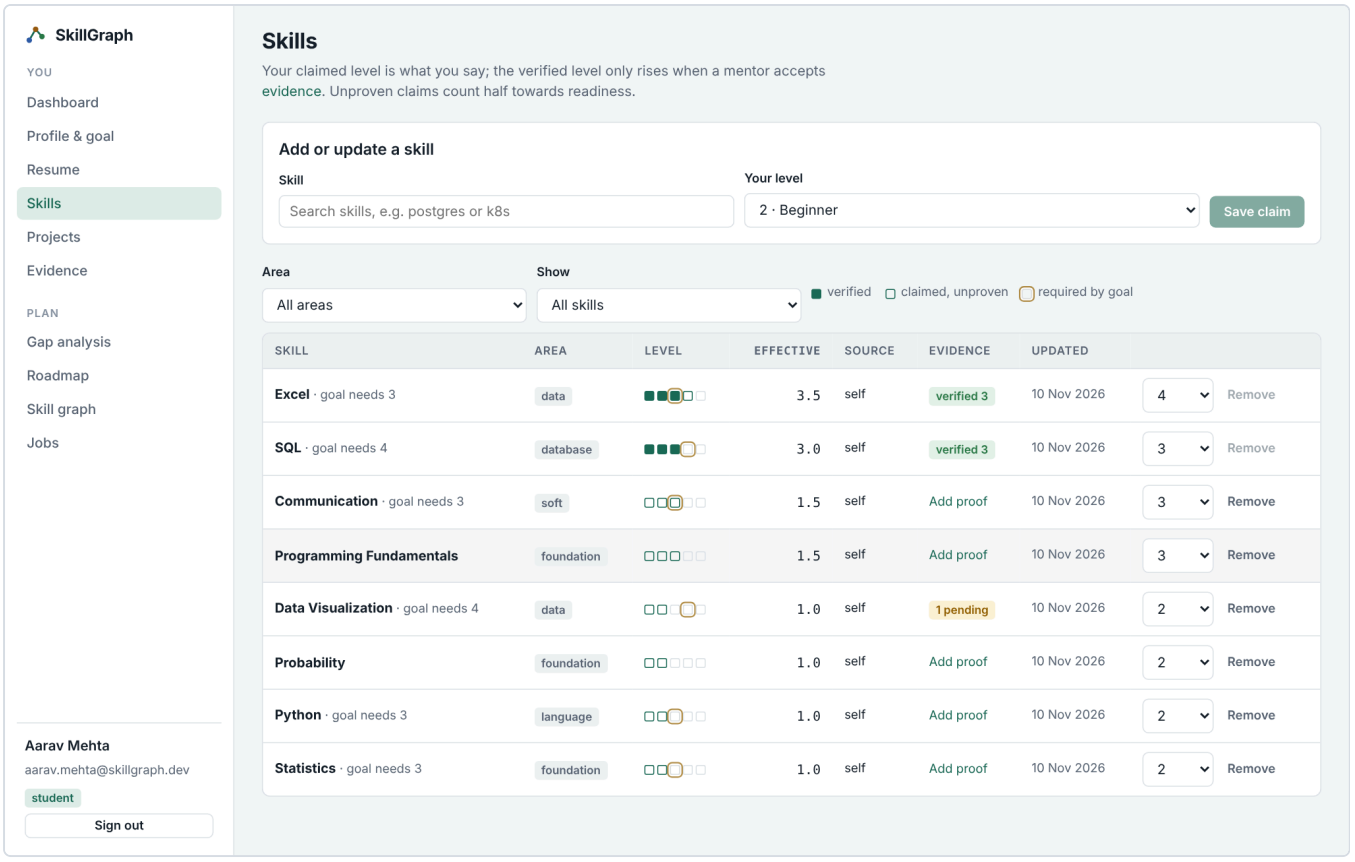
Student dashboard: readiness, next step, top priorities, job matches, claims vs. evidence, MongoDB activity



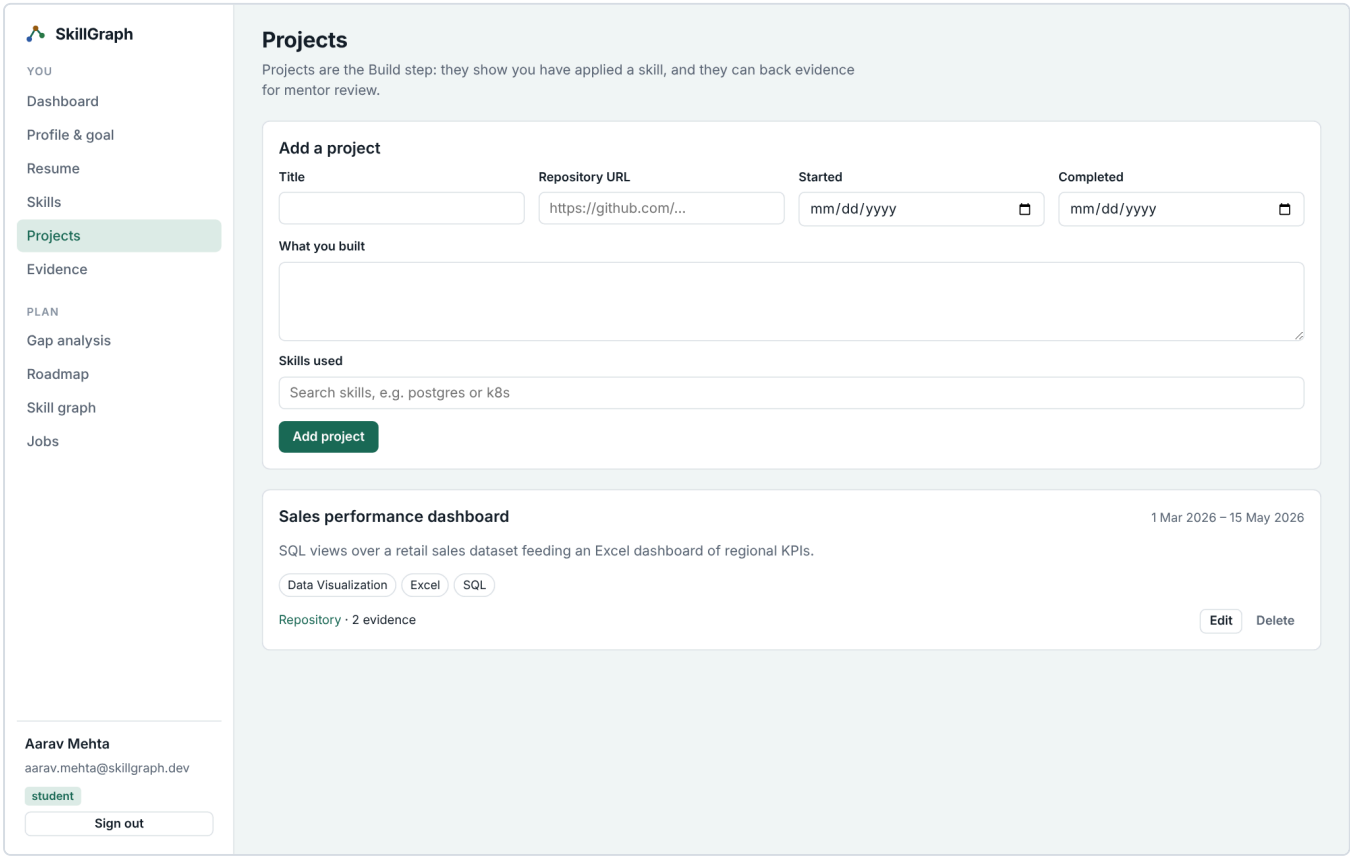
Profile and target role ranked by pgvector profile fit



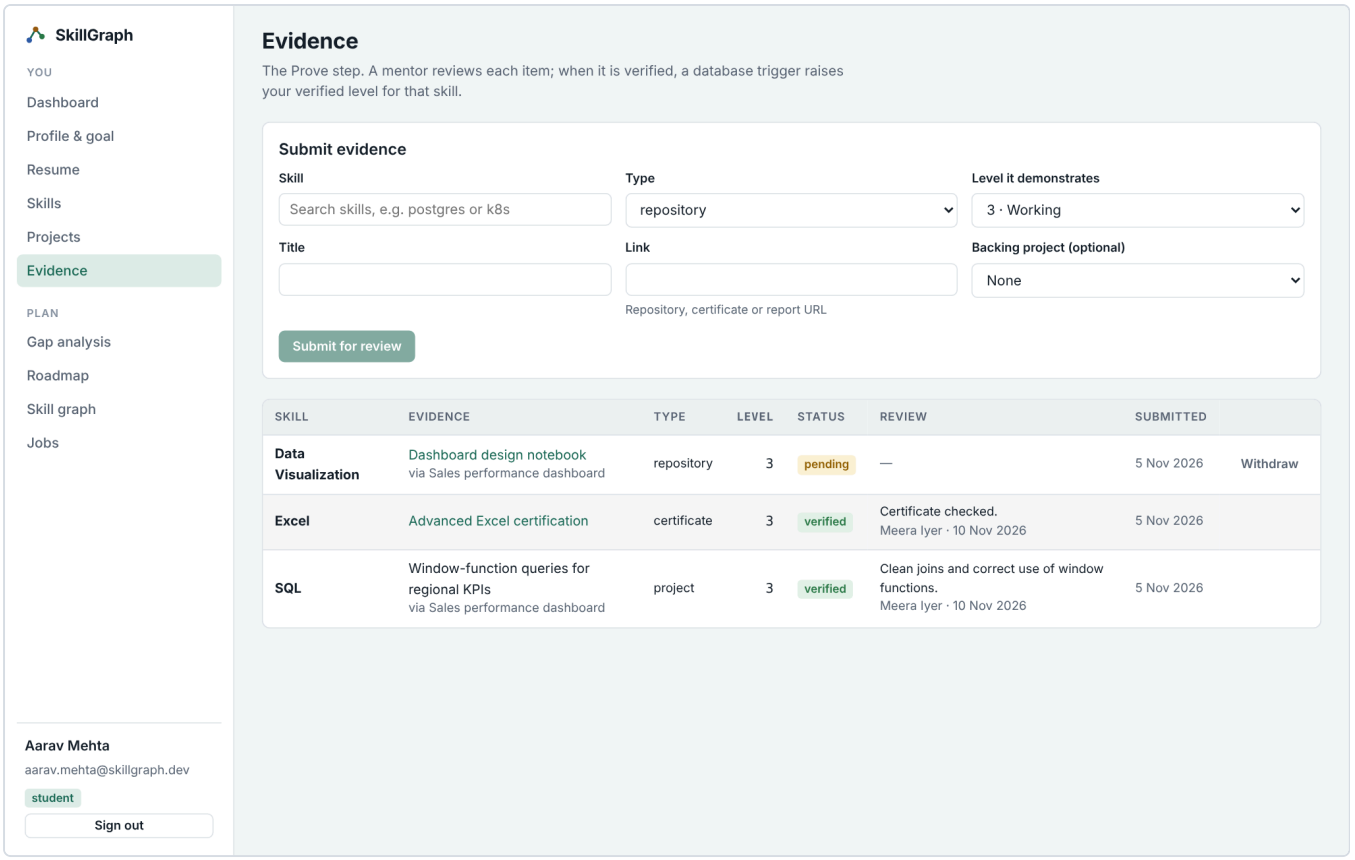
Resume: skills detected from the MongoDB document, levels to import, catalogue gaps



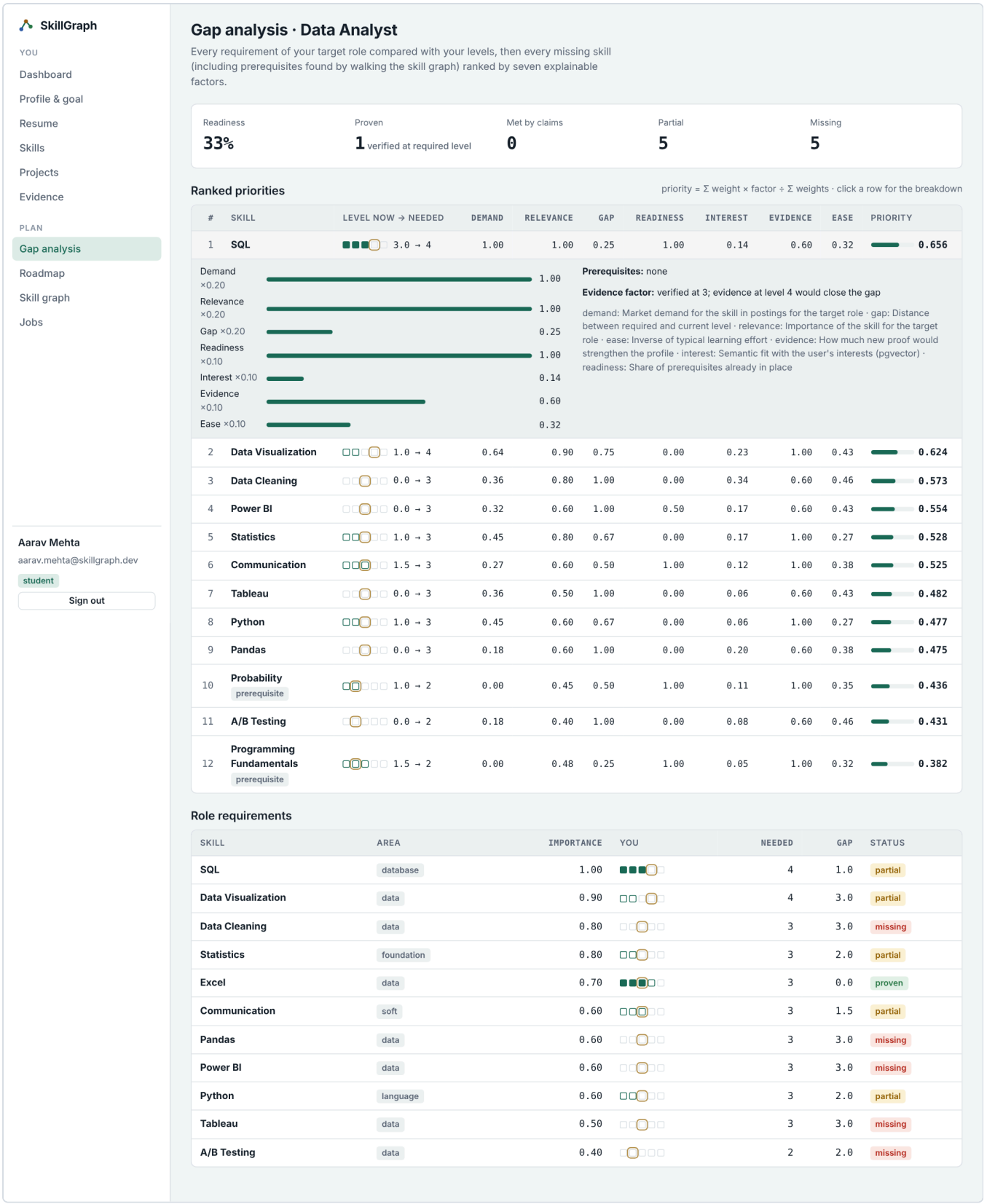
Skills: level pips show verified (solid), unproven claims (hollow) and the goal requirement (ring)



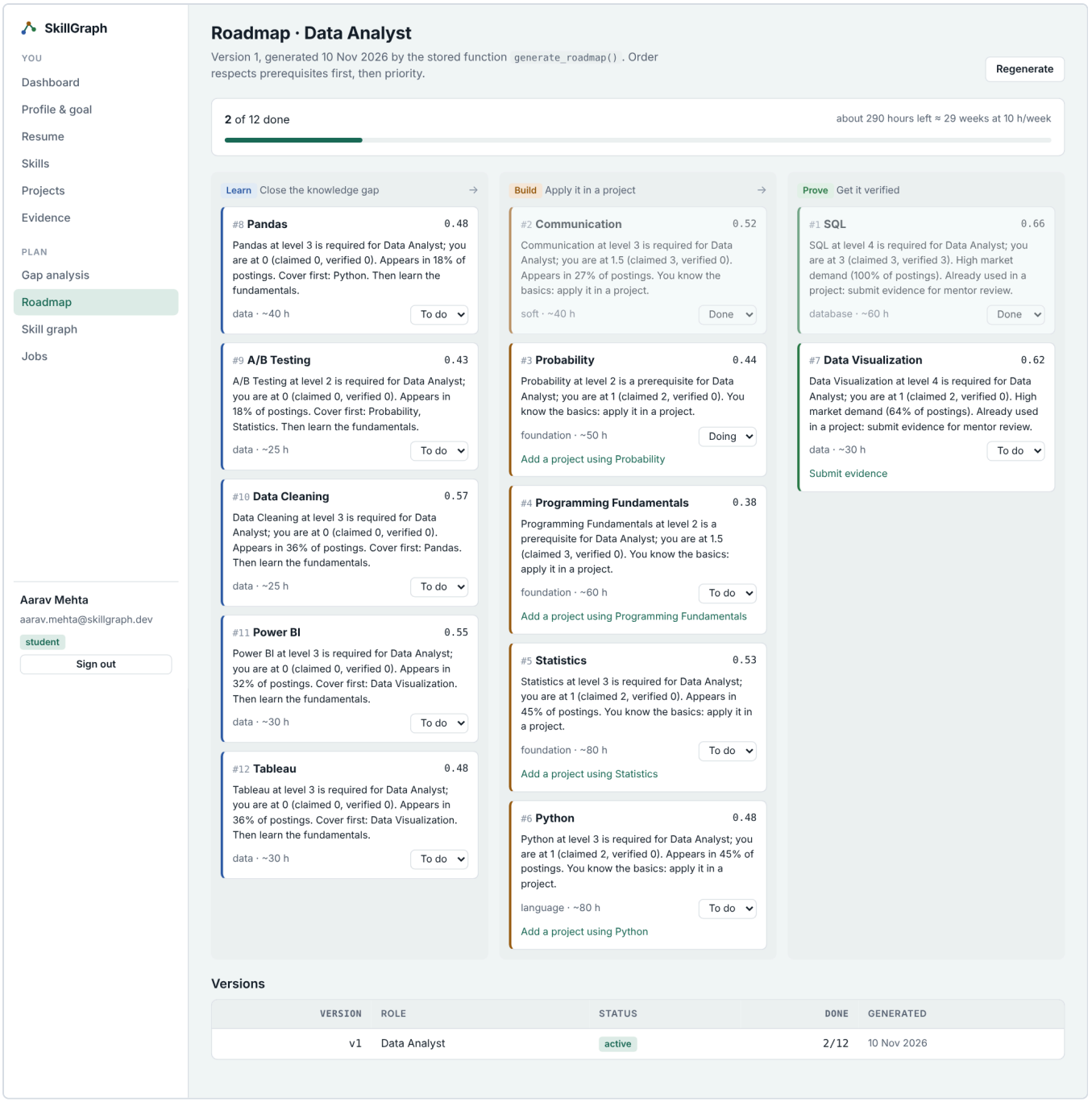
Projects



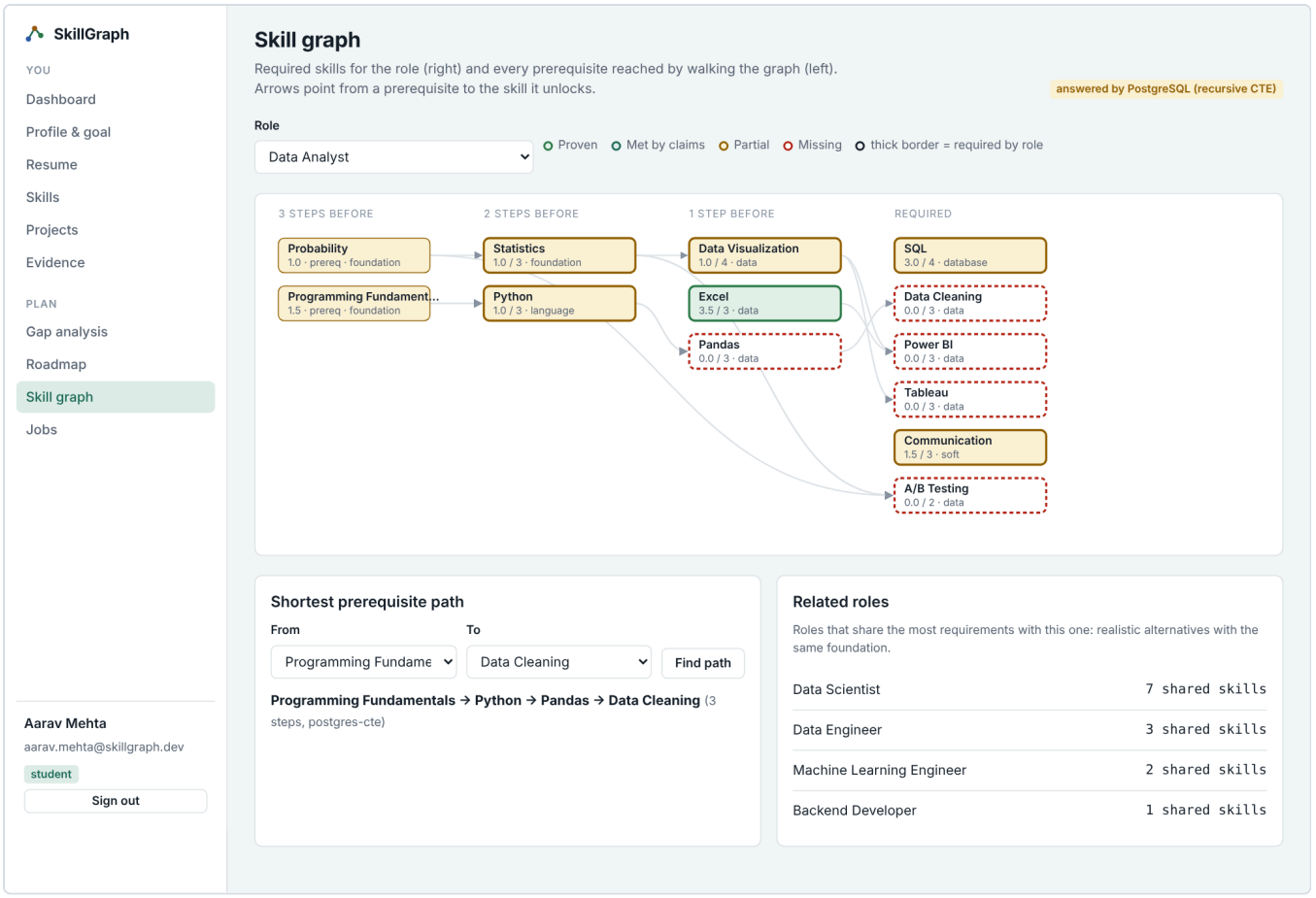
Evidence submission and review status



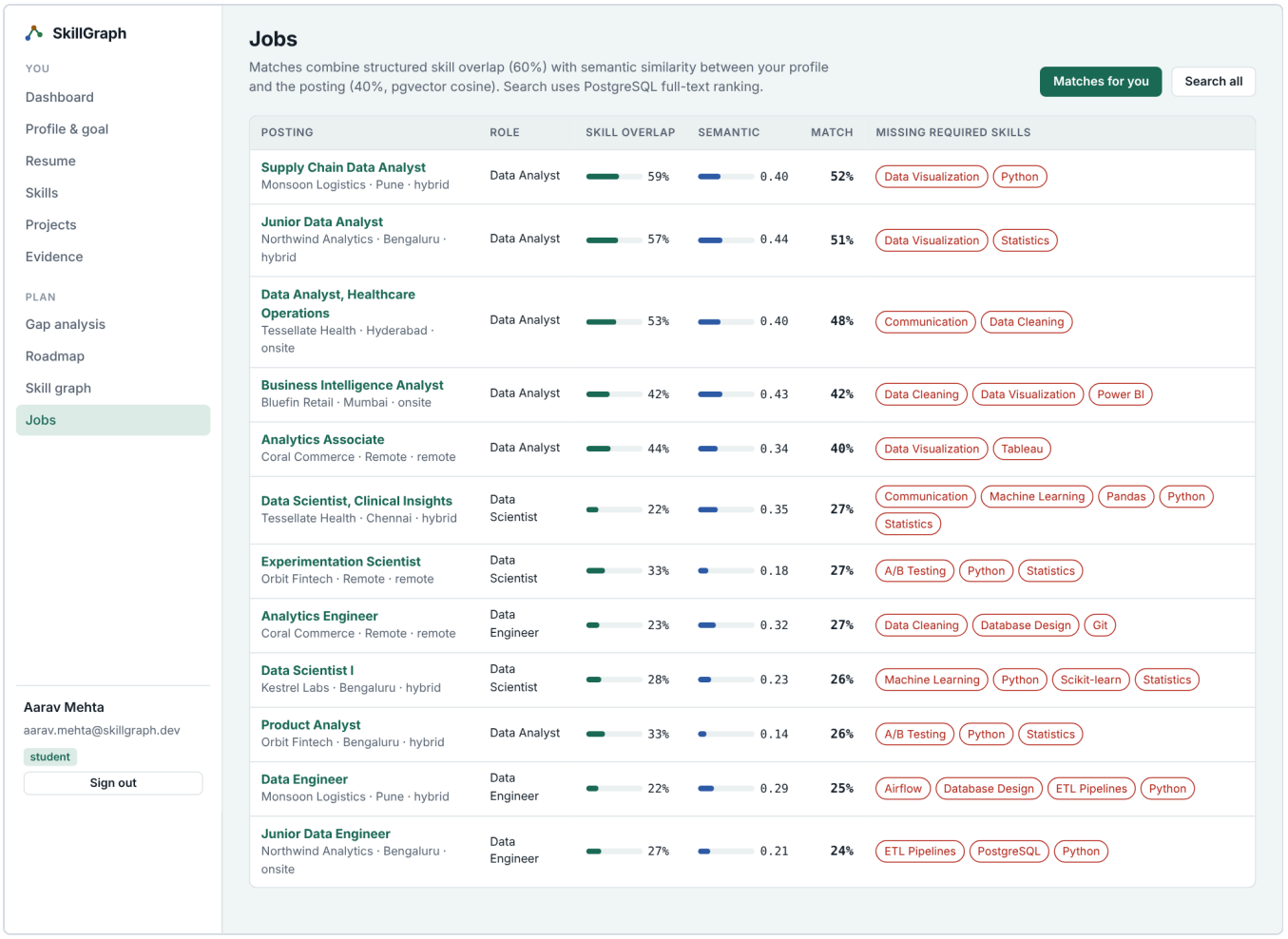
Gap analysis: 7-factor ranking with an expanded breakdown



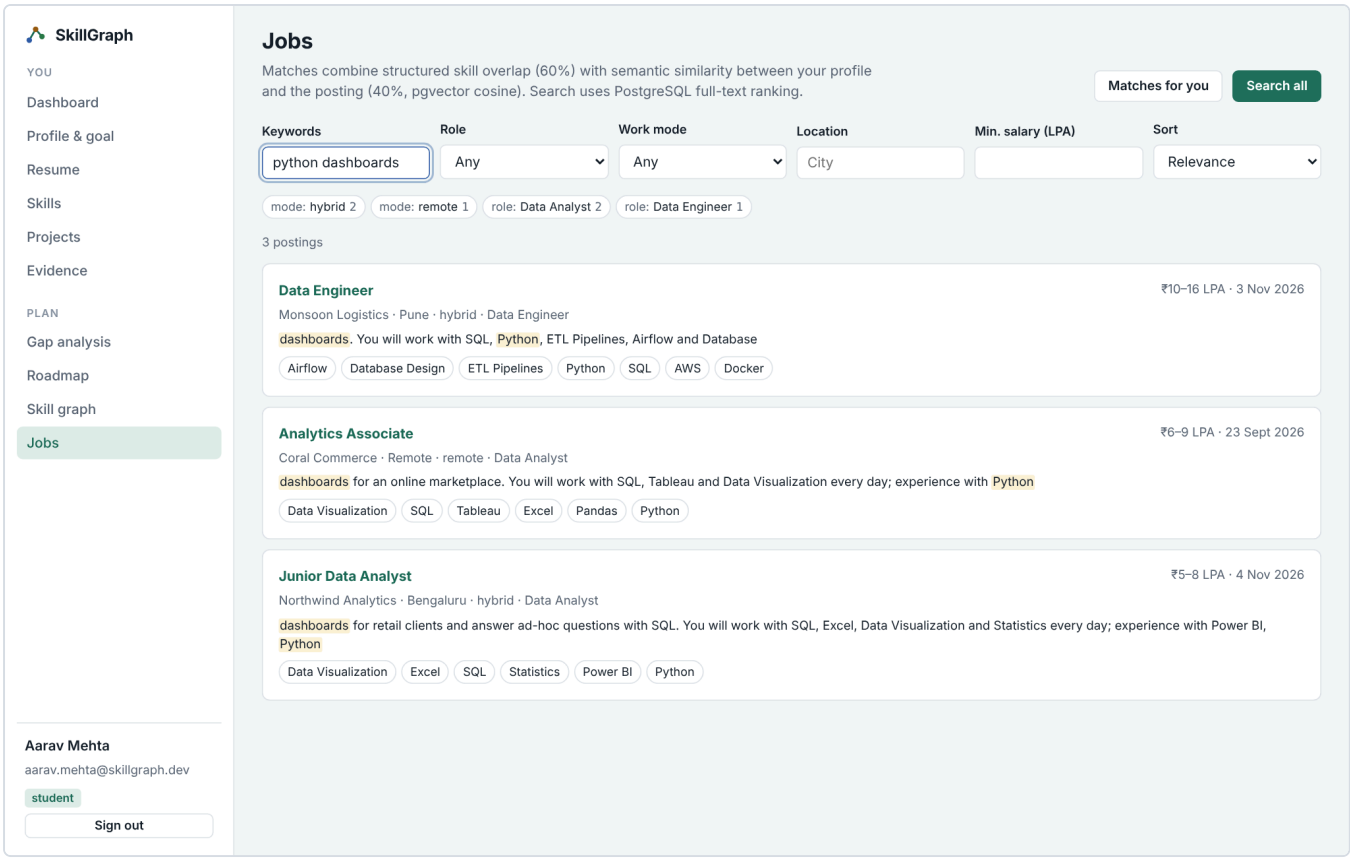
Roadmap: Learn → Build → Prove lanes with generated rationale and version history



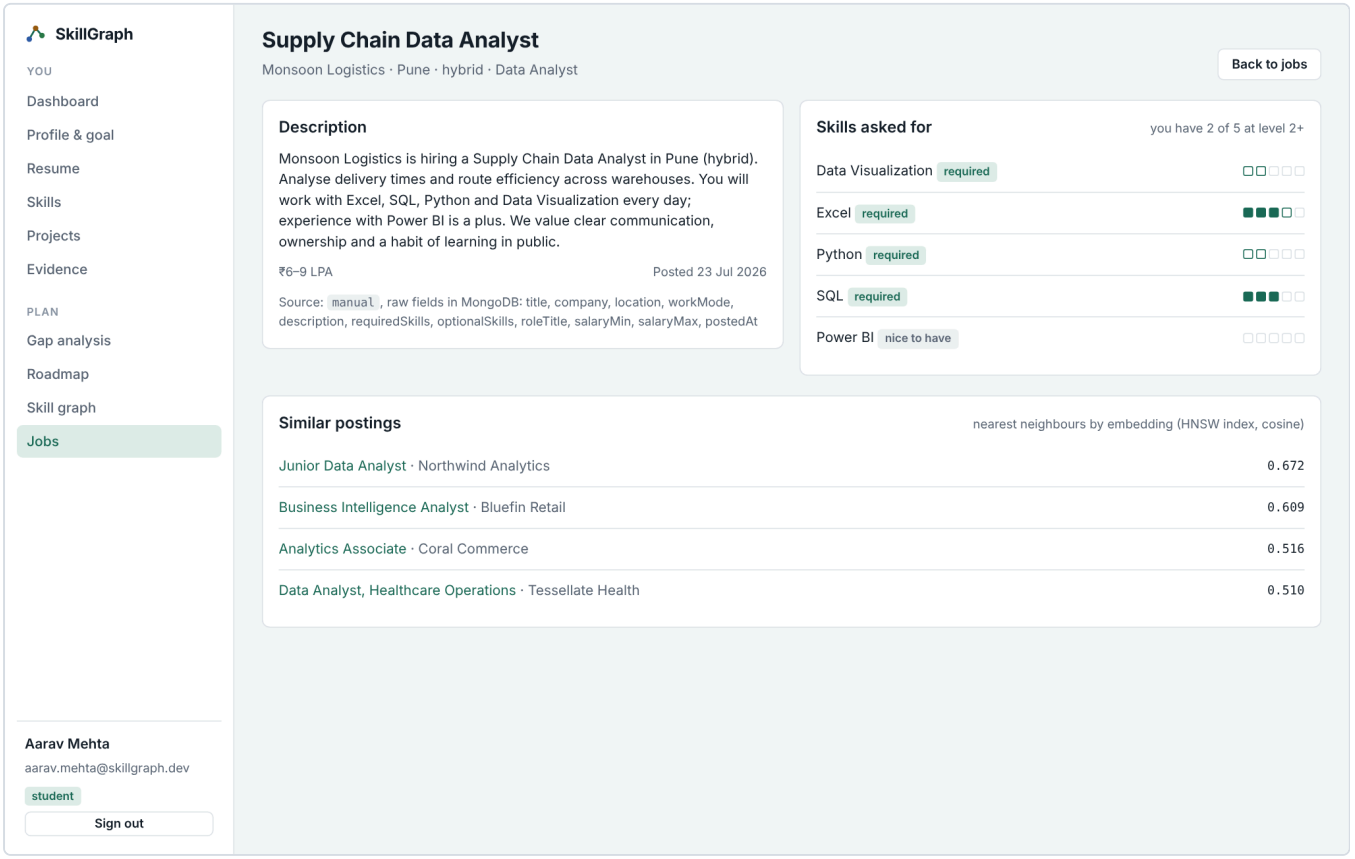
Skill graph with shortest prerequisite path and related roles



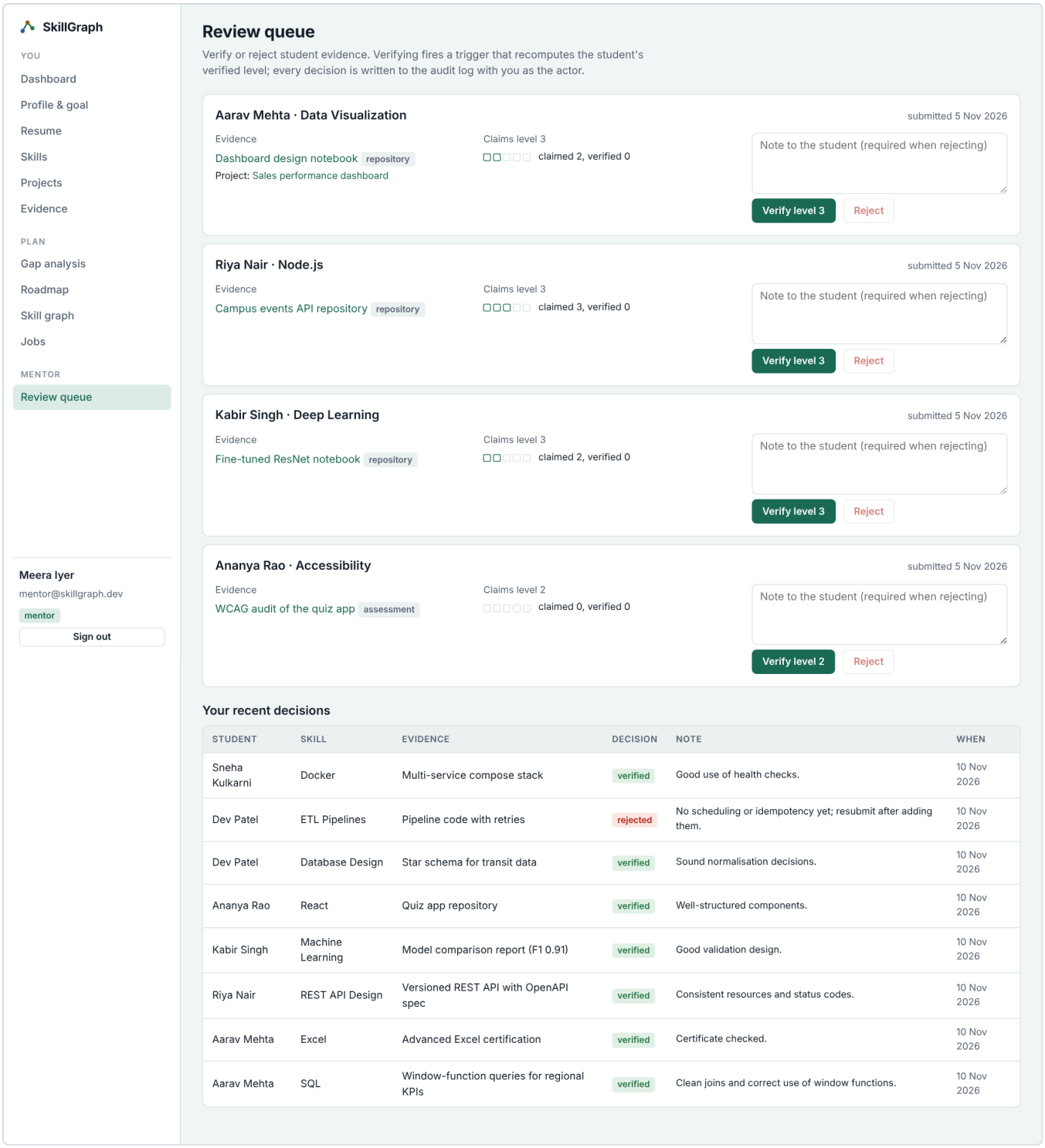
Job matches: skill overlap + semantic similarity



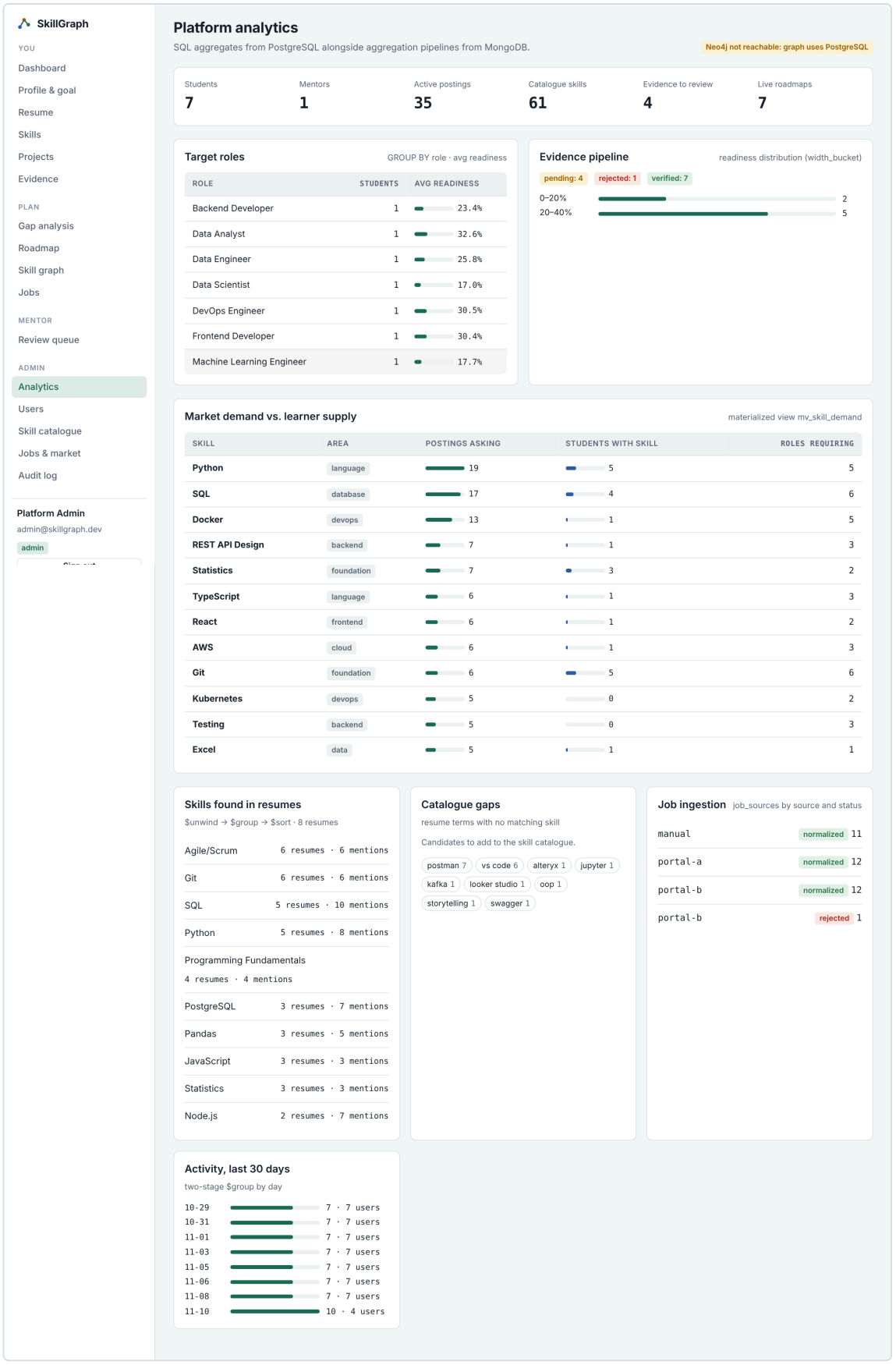
Job search: full-text ranking, highlighted snippets, facets



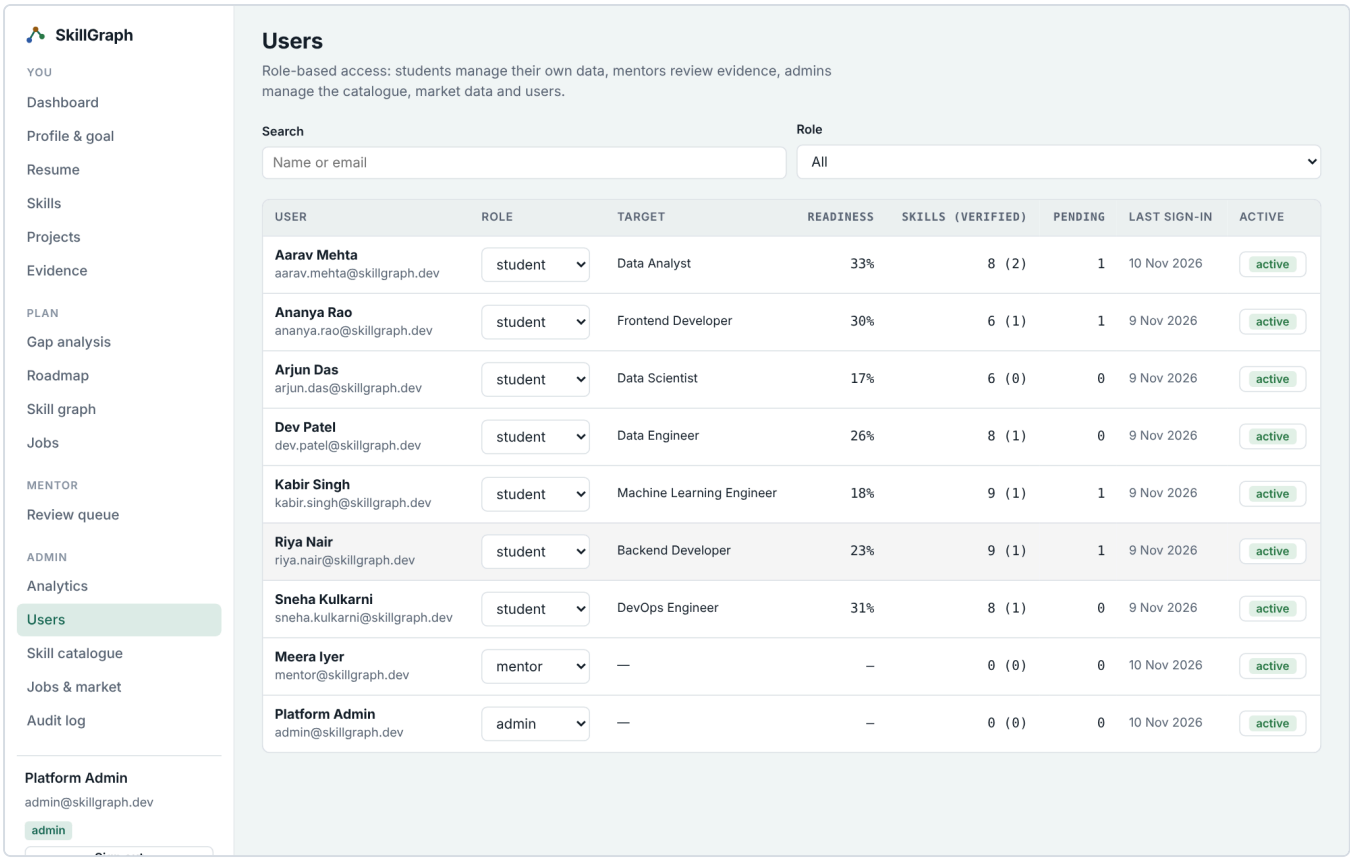
Job detail: required skills vs. your levels, similar postings by vector k-NN, MongoDB provenance



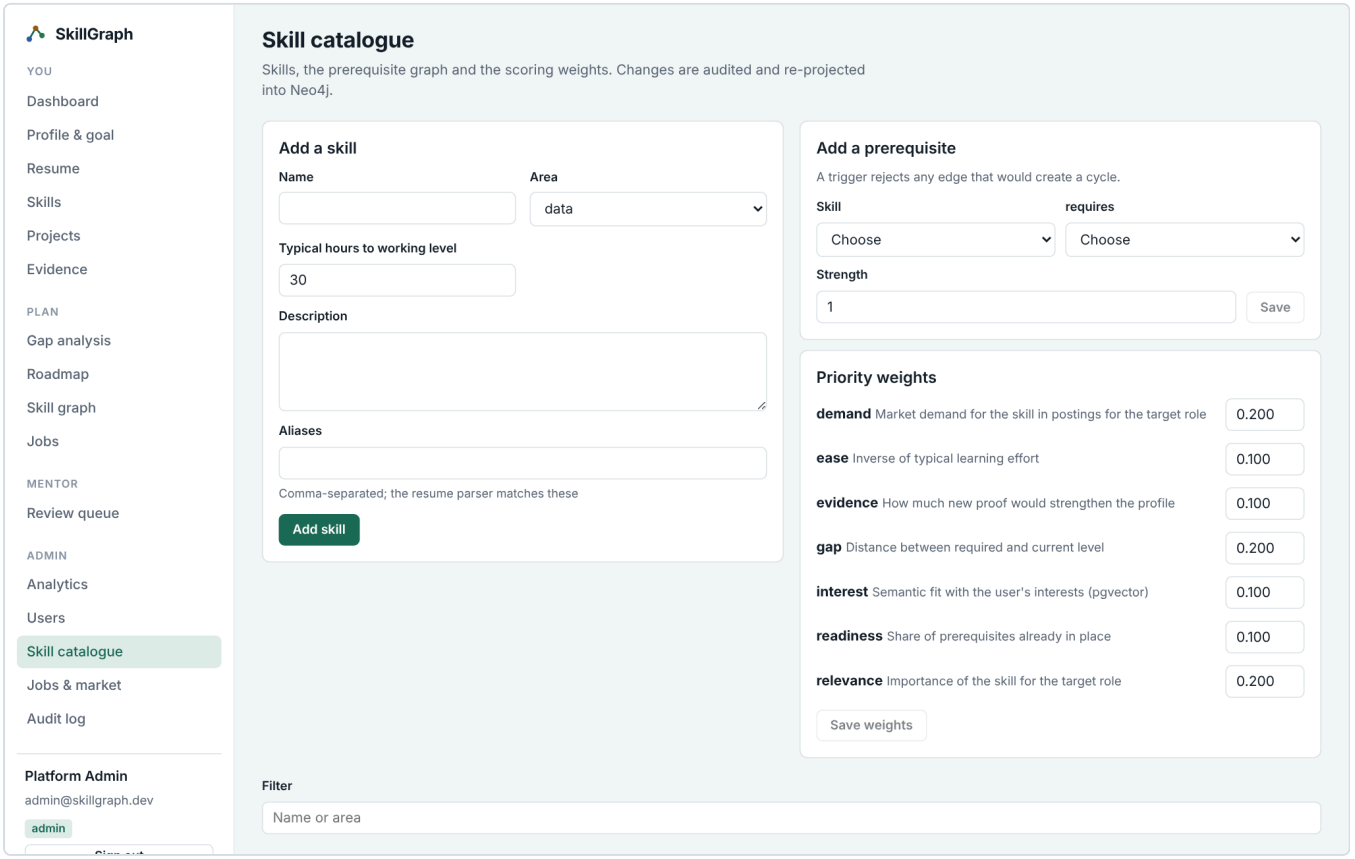
Mentor review queue



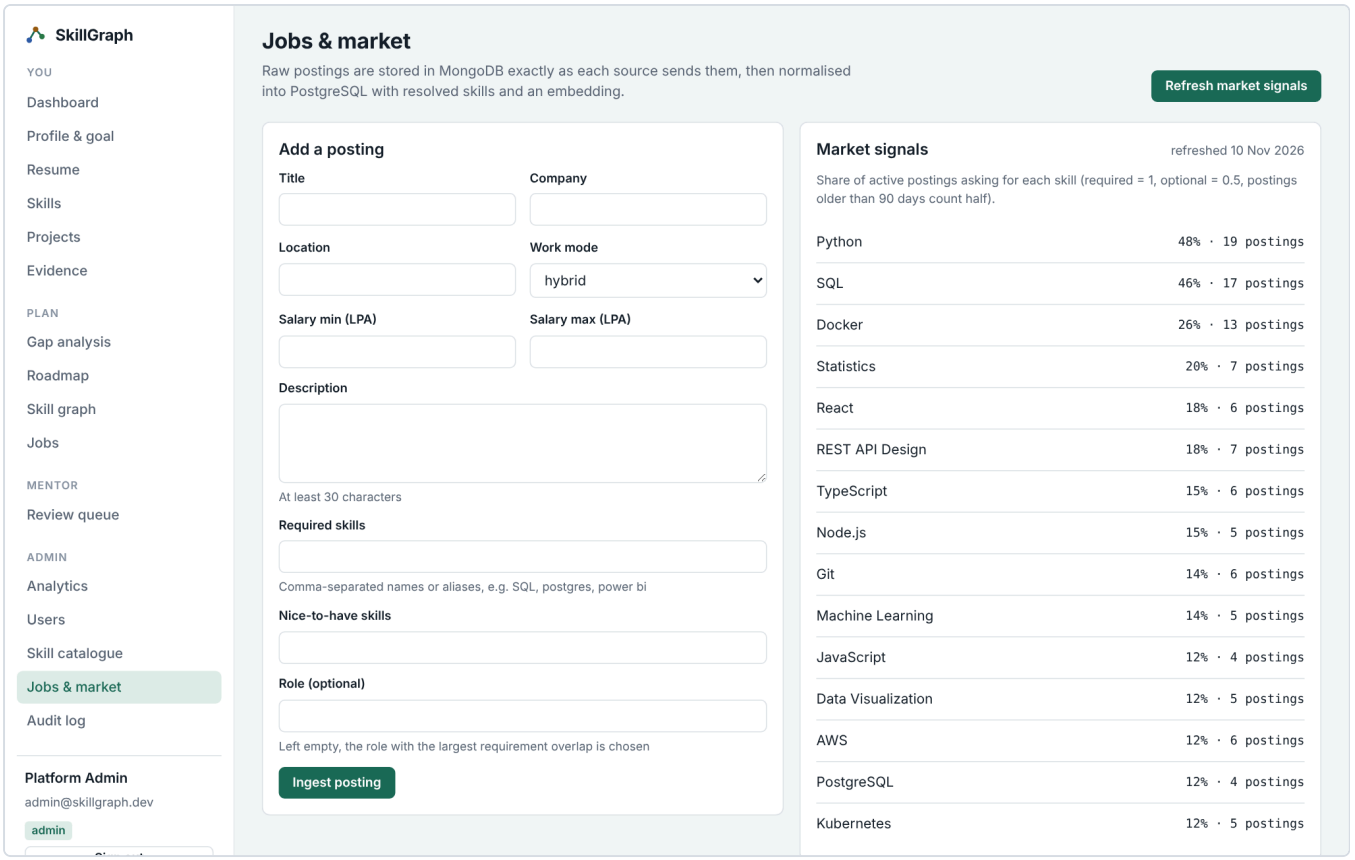
Admin analytics: SQL aggregates and MongoDB aggregation pipelines



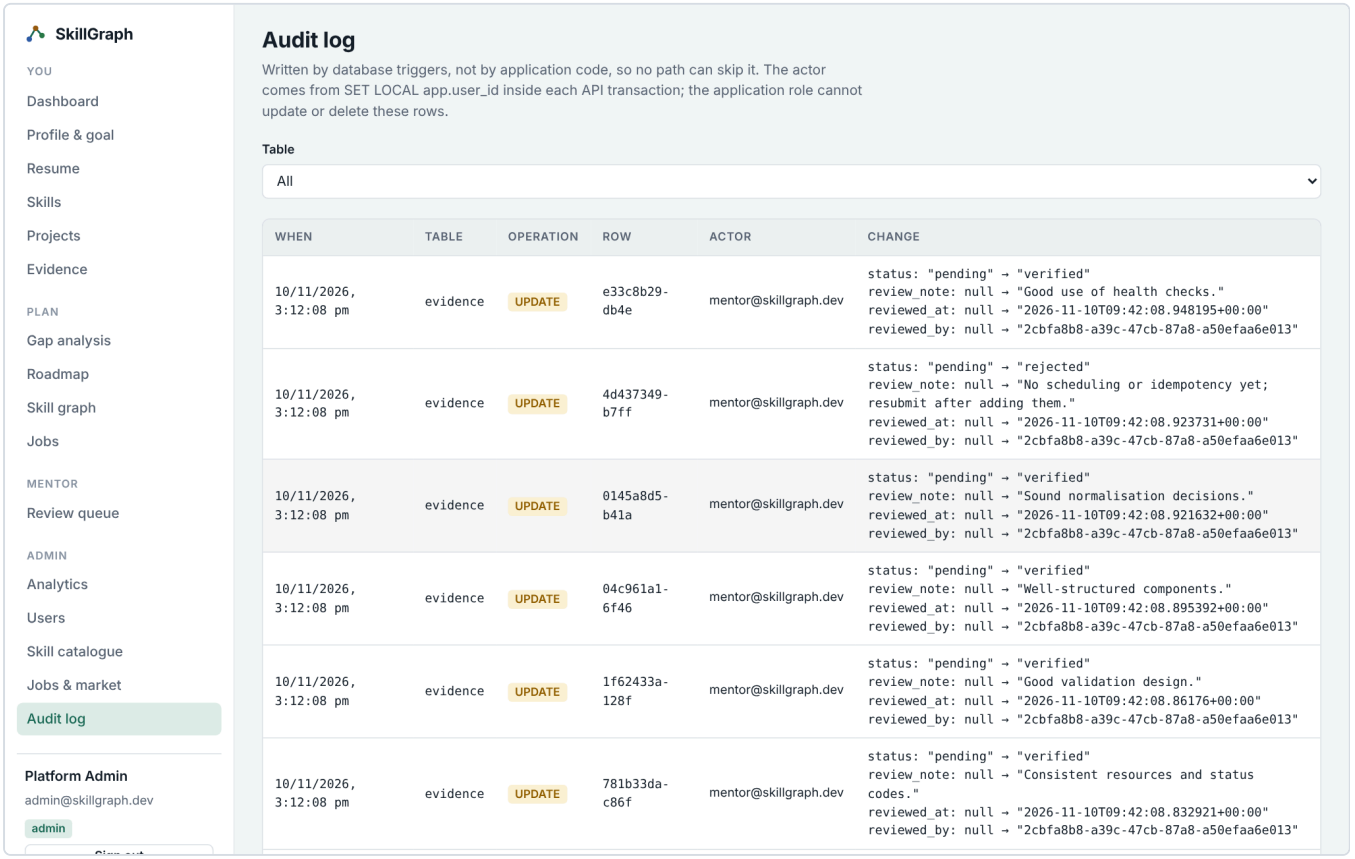
Admin users and roles



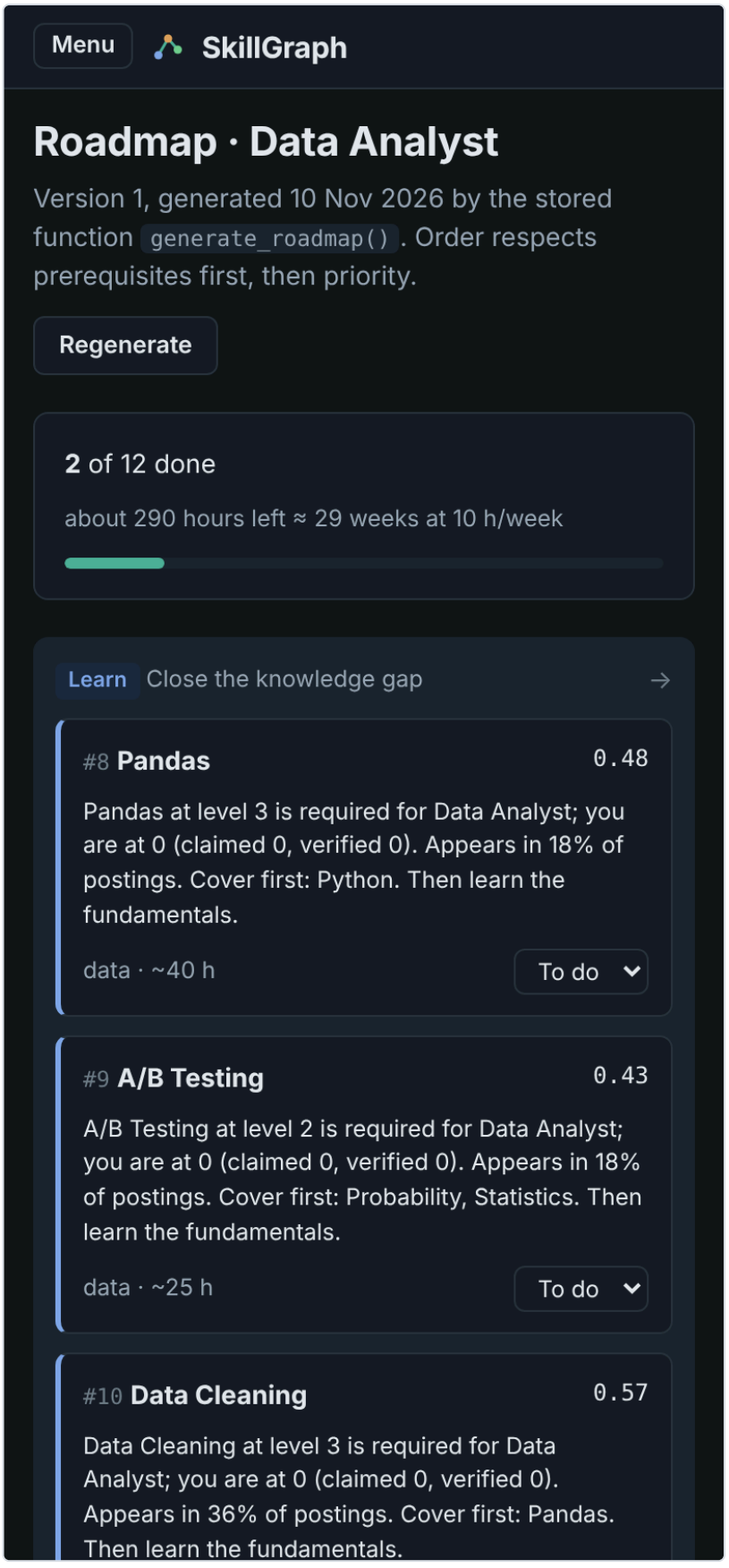
Skill catalogue, prerequisites and scoring weights



Job ingestion and market signals



Audit log written by triggers



Phone width, dark mode

13. Testing results

Run `npm test`. The suite has **50 automated tests in 9 groups, and all 50 pass.**

- The database tests run inside rolled-back transactions as the application role.
- The API tests drive the running app over HTTP with real sessions and clean up afterwards in both PostgreSQL and MongoDB.

Database tests (25)

AREA	TEST	RESULT
Constraints	proficiency domain rejects level 6 · malformed email rejected · plain-text password rejected (bcrypt CHECK) · case-insensitive unique email (citext) · completion date before start rejected · evidence needs a link or project · FK RESTRICT blocks deleting a used skill · ON DELETE CASCADE removes dependants	8 / 8 pass
Triggers	new goal retires the old one · verification raises verified_level and lifts the claim, deleting evidence removes it · a student cannot review, a mentor cannot self-review · evidence cannot use another student's project · prerequisite cycle rejected · audit row records actor and never leaks `password_hash`	6 / 6 pass
Functions	priorities: only real gaps, all factors in [0,1], transitive prerequisites included · changing weights changes ranking · roadmap versions + no prerequisite after its dependent · no goal → P0002 · all items done → roadmap completed · match_jobs scores in [0,1] sorted · procedure runs as app role	7 / 7 pass
Indexes	FTS query plan uses `ix_jobs_fts` · k-NN plan uses HNSW `ix_jobs_embedding`	2 / 2 pass
Privileges	app role cannot CREATE/DROP tables · cannot UPDATE/DELETE audit_log	2 / 2 pass

API end-to-end tests (25)

AREA	TEST	RESULT
Auth	401 without session · field-level validation errors · register + auto sign-in · duplicate email 409 · identical error for wrong password vs. unknown email · mentor and admin sign-in	6 / 6 pass
RBAC	student blocked from mentor and admin APIs · mentor blocked from admin APIs	2 / 2 pass
Student journey	goal → roadmap in one transaction · PDF upload parsed (≥ 10 skills, unknown terms) · fake PDF rejected (415) · import is atomic (invalid id rolls back everything) · regenerate → v2, v1 archived · evidence needs link or project · mentor verify → verified level 3, double review 409, cannot withdraw verified · audit shows mentor as actor · gap, graph, matches respond · **SQL injection** strings treated as text · search filters and sorting	11 / 11 pass
Admin	unknown-only skills rejected, valid posting normalised with unresolved terms reported · market refresh procedure · prerequisite cycle → 422 · admin cannot demote self · analytics combine SQL + MongoDB · deactivation ends access immediately	6 / 6 pass

Other verification

- `next build` and `tsc` complete with no type errors.
- All 15 demo queries, the DDL demo and the transaction demo run cleanly.
- `aggregations.js` runs in `mongosh` against MongoDB.
- All 32 Cypher statements pass the official Cypher syntax validator (`@neo4j-cypher/language-support`).
- `npm audit` reports 0 vulnerabilities.

- The UI was checked at desktop and phone width (no horizontal scroll) in light and dark mode.

Test environment. PostgreSQL 16 + pgvector 0.6 and MongoDB 8.3 (compose uses MongoDB 7). Graph features ran on the PostgreSQL fallback path. Before the demo, run `npm test` and `npm run graph:sync` with the full Docker stack: the graph page badge should read "Neo4j (Cypher)".

14. Conclusion and future enhancements

SkillGraph shows that the database can do the system's reasoning, not just its storage:

- Integrity rules (who may verify evidence, an acyclic prerequisite graph, one active goal, derived verified levels) live in constraints and triggers, so no API path can bypass them.
- Ranking and roadmap generation are set-based SQL over recursive CTEs, run in transactions.
- MongoDB keeps raw, irregular documents and analyses them with aggregation pipelines.
- Neo4j answers multi-hop career questions.
- pgvector adds meaning-level matching inside ordinary SQL joins.

Every recommendation can be traced back to stored data, through `score_breakdown` and `rationale`.

Future enhancements

1. LLM-assisted resume parsing, with the current rules kept as a validator.
2. Scheduled job ingestion from public APIs, and Neo4j change-data-capture instead of full re-projection.
3. Learning-resource recommendations per roadmap item (the `learning_resources` table from the DDL demo).
4. Row-level security policies as a second layer under the API checks.
5. Mentor assignment and comment threads on evidence.
6. Calibrating the scoring weights from outcome data (who got hired after which roadmap).
7. OCR for scanned resumes.

15. Team contribution

AREA	ADITYA KALLOLI (PES1UG24AM015)	ABHAY BHARADWAJ (PES1UG24AM007)
Database	PostgreSQL schema, constraints, domains, indexes; priority, roadmap and match functions; triggers; procedure; pgvector	MongoDB schema, validators, aggregations, ingestion pipeline; Neo4j model, sync and Cypher; recursive-CTE fallback
Application	Project scaffold and UI shell; gap, roadmap, jobs, skills, projects, evidence, profile and dashboard (APIs and pages); embeddings; seed data	Auth, RBAC, middleware; resume parsing and upload; mentor and admin (APIs and pages); skill-graph page
Quality	Database test suite (25 tests); SQL demo scripts	API end-to-end tests (25 tests); security review; report, ER diagram, data dictionary and screenshots

Each member commits their own work from their own Git identity, so the repository history shows these contributions.

Appendix A. Requirement compliance

#	REQUIREMENT	WHERE IT IS MET
1	Unique title, real problem, not CRUD	SkillGraph: explainable prioritisation, roadmap generation, hybrid matching
2	Two-member team, both on database and app	§15
3	SQL + NoSQL / vector	PostgreSQL + MongoDB + Neo4j + pgvector
4	ER diagram → relational schema with entities, attributes, PK/FK, relationships, constraints; NoSQL/vector data model	§5, §6, §8, §9.2–9.4, `DATA_DICTIONARY.md`
5	DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	`01_schema.sql`, `02_logic.sql`, `demo/01_ddl_alter_drop.sql`
6	DML with JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	API routes; `demo/02_important_queries.sql`; `demo/03_transactions_dml.sql`
7	≥ 4 advanced features	Transactions, views (+ materialized), stored functions + procedure, triggers, indexing (B-tree, partial, GIN, trigram, HNSW), full-text search, NoSQL aggregation, vector similarity search: 8 of 8
8	Vector search explained	§9.4
9	Next.js interface with forms, tables, search, filters, dashboards	§12 (20 screens)
10	API layer, env-based config	`app/api/*`, `lib/env.ts`, `.env.example`
11	Authentication, ≥ 2 roles	student, mentor, admin
12	Database-level validation and integrity	4 domains, 51 CHECKs, 26 FKs, triggers, partial unique indexes; Mongo `$jsonSchema`
13	Security	§11 security summary, `README.md`
14	Documentation	This report
15	Git/GitHub, README	`README.md`, §15
16	Demonstration	Appendix B
17	Originality	Original design and code
18	Level 3, justified database choices	§7

Appendix B. Demonstration script (about 12 minutes)

1. **Problem (1 min).** Dashboard as Aarav: readiness 33%, "Do next", claims vs. evidence.
2. **ER and schema (2 min).** ER diagram; `\d evidence` in psql (constraints and triggers); `demo/01_ddl_alter_drop.sql`.
3. **Resume → MongoDB (1.5 min).** Upload `samples/sample_resume_neha_joshi.pdf` as a new student, import the skills (one transaction), show the document in `mongosh`.
4. **Reasoning in SQL (2 min).** Gap page with an expanded breakdown; `SELECT * FROM compute_skill_priorities(…)`; regenerate the roadmap (v2) and show the version history.
5. **Claims → evidence (1.5 min).** Submit evidence as the student; verify as the mentor; the verified level rises; show the audit log.
6. **Graph (1 min).** Skill graph page with the Neo4j badge; shortest path; G1/G2 in Neo4j Browser.
7. **Vector + full-text (1 min).** Job matches (semantic column); job detail "similar postings"; search "python dashboards".
8. **Admin (1 min).** Ingest a posting with an unknown skill; refresh market signals; analytics (MongoDB catalogue gaps).
9. **Tests (1 min).** `npm test`: 50 passing.

Appendix C. Data dictionary (generated)

Generated from the PostgreSQL catalogue by `docs/tools/data_dictionary.py`. Key: PK primary key, FK foreign key, UK unique, NN not null, GEN generated column.

app_meta

Key/value metadata (schema version, embedding model, last market refresh).

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`key`	character varying(40)	PK	NN	
`value`	text		NN	

audit_log

Append-only change history written by triggers.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`audit_id`	bigint	PK	NN	nextval('audit_log_audit_id_seq'::regclass)
`table_name`	character varying(40)		NN	
`operation`	character varying(6)		NN	
`row_pk`	text		NN	
`actor_id`	uuid		yes	
`old_data`	jsonb		yes	
`new_data`	jsonb		yes	
`changed_at`	timestamp with time zone		NN	now()

Constraints

- `audit_log_operation_check: CHECK (((operation)::text = ANY ((ARRAY['INSERT'::character varying, 'UPDATE'::character varying, 'DELETE'::character varying])::text[])))`

career_roles

Catalogue of target roles.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`role_id`	integer	PK	NN	nextval('career_roles_role_id_seq'::regclass)
`title`	character varying(100)	UK	NN	
`domain`	character varying(60)		NN	
`seniority`	character varying(10)		NN	'junior'::character varying
`description`	text		NN	
`neo4j_node_id`	character varying(64)		yes	
`embedding`	vector(384)		yes	
`created_at`	timestamp with time zone		NN	now()

Constraints

- `career_roles_seniority_check`: `CHECK (((seniority)::text = ANY ((ARRAY['intern'::character varying, 'junior'::character varying, 'mid'::character varying, 'senior'::character varying])::text[])))`

evidence

Proof of a skill, reviewed by a mentor.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`evidence_id`	uuid	PK	NN	gen_random_uuid()
`user_id`	uuid	FK	NN	→ users.user_id
`skill_id`	integer	FK	NN	→ skills.skill_id
`project_id`	uuid	FK	yes	→ projects.project_id
`kind`	evidence_kind		NN	
`title`	character varying(150)		NN	
`url`	web_url		yes	
`demonstrated_level`	proficiency		NN	
`status`	character varying(10)		NN	'pending'::character varying
`reviewed_by`	uuid	FK	yes	→ users.user_id
`reviewed_at`	timestamp with time zone		yes	
`review_note`	character varying(500)		yes	
`submitted_at`	timestamp with time zone		NN	now()

Constraints

- `chk_has_backing`: `CHECK (((url IS NOT NULL) OR (project_id IS NOT NULL)))`
- `chk_review_state`: `CHECK ((((status)::text = 'pending'::text) = (reviewed_at IS NULL)))`
- `evidence_demonstrated_level_check`: `CHECK (((demonstrated_level)::smallint >= 1))`
- `evidence_status_check`: `CHECK (((status)::text = ANY ((ARRAY['pending'::character varying, 'verified'::character varying, 'rejected'::character varying])::text[])))`
- `evidence_title_check`: `CHECK ((length(btrim((title)::text)) >= 3))`
- `evidence_project_id_fkey`: `FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE SET NULL`
- `evidence_reviewed_by_fkey`: `FOREIGN KEY (reviewed_by) REFERENCES users(user_id) ON DELETE SET NULL`
- `evidence_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT`
- `evidence_user_id_fkey`: `FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE`

goals

Target career role chosen by a user (one active at a time).

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`goal_id`	uuid	PK	NN	gen_random_uuid()
`user_id`	uuid	FK	NN	→ users.user_id
`role_id`	integer	FK	NN	→ career_roles.role_id
`target_date`	date		yes	
`status`	character varying(10)		NN	'active'::character varying

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
created_at	timestamp with time zone		NN	now()

Constraints

- goals_status_check: CHECK (((status)::text = ANY ((ARRAY['active'::character varying, 'achieved'::character varying, 'dropped'::character varying])::text[])))
- goals_target_date_check: CHECK ((target_date >= '2024-01-01'::date))
- goals_role_id_fkey: FOREIGN KEY (role_id) REFERENCES career_roles(role_id) ON DELETE RESTRICT
- goals_user_id_fkey: FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE

job_postings

Normalised job postings (raw form in MongoDB).

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
job_id	uuid	PK	NN	gen_random_uuid()
role_id	integer	FK	yes	→ career_roles.role_id
mongo_source_id	character(24)	UK	yes	
title	character varying(150)		NN	
company	character varying(120)		NN	
location	character varying(120)		NN	
work_mode	character varying(10)		NN	'onsite'::character varying
description	text		NN	
salary_min_lpa	numeric(5,1)		yes	
salary_max_lpa	numeric(5,1)		yes	
posted_at	date		NN	CURRENT_DATE
is_active	boolean		NN	true
embedding	vector(384)		yes	
search_tsv	tsvector		yes	GEN (tsvector)
created_at	timestamp with time zone		NN	now()

Constraints

- chk_salary_range: CHECK (((salary_max_lpa IS NULL) OR (salary_min_lpa IS NULL) OR (salary_max_lpa >= salary_min_lpa)))
- job_postings_description_check: CHECK ((length(description) >= 30))
- job_postings_mongo_source_id_check: CHECK ((mongo_source_id ~ '^[0-9a-f]{24}$'::text))
- job_postings_salary_min_lpa_check: CHECK ((salary_min_lpa > (0)::numeric))
- job_postings_work_mode_check: CHECK (((work_mode)::text = ANY ((ARRAY['onsite'::character varying, 'remote'::character varying, 'hybrid'::character varying])::text[])))
- job_postings_role_id_fkey: FOREIGN KEY (role_id) REFERENCES career_roles(role_id) ON DELETE SET NULL

job_skills

M:N: skills a posting asks for.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
job_id	uuid	FK, PK	NN	→ job_postings.job_id
skill_id	integer	FK, PK	NN	→ skills.skill_id
is_required	boolean		NN	true

Constraints

- job_skills_job_id_fkey: FOREIGN KEY (job_id) REFERENCES job_postings(job_id) ON DELETE CASCADE
- job_skills_skill_id_fkey: FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT

market_signals

Daily demand snapshot per skill and role.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
signal_id	bigint	PK	NN	nextval('market_signals_signal_id_seq'::regclass)
skill_id	integer	FK, UK	NN	→ skills.skill_id
role_id	integer	FK, UK	yes	→ career_roles.role_id
demand_score	unit_score		NN	
posting_count	integer		NN	
snapshot_date	date	UK	NN	CURRENT_DATE

Constraints

- market_signals_posting_count_check: CHECK ((posting_count >= 0))
- market_signals_role_id_fkey: FOREIGN KEY (role_id) REFERENCES career_roles(role_id) ON DELETE CASCADE
- market_signals_skill_id_fkey: FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE

profiles

Personal profile, 1:1 with users; holds the profile embedding.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
profile_id	uuid	PK	NN	gen_random_uuid()
user_id	uuid	FK, UK	NN	→ users.user_id
full_name	character varying(120)		NN	
education_level	character varying(30)		yes	
institution	character varying(150)		yes	
graduation_year	smallint		yes	
interests	text[]		NN	'{}'::text[]
bio	text		yes	
weekly_hours	smallint		NN	8
embedding	vector(384)		yes	
updated_at	timestamp with time zone		NN	now()

Constraints

- `profiles_bio_check`: `CHECK ((length(bio) <= 1000))`
- `profiles_education_level_check`: `CHECK (((education_level)::text = ANY ((ARRAY['high_school'::character varying, 'diploma'::character varying, 'bachelors'::character varying, 'masters'::character varying, 'phd'::character varying, 'other'::character varying])::text[])))`
- `profiles_full_name_check`: `CHECK ((length(btrim((full_name)::text)) >= 2))`
- `profiles_graduation_year_check`: `CHECK (((graduation_year >= 1980) AND (graduation_year <= 2040)))`
- `profiles_interests_check`: `CHECK ((cardinality(interests) <= 15))`
- `profiles_weekly_hours_check`: `CHECK (((weekly_hours >= 1) AND (weekly_hours <= 80)))`
- `profiles_user_id_fkey`: `FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE`

project_skills

M:N: skills used in a project.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`project_id`	uuid	FK, PK	NN	→ projects.project_id
`skill_id`	integer	FK, PK	NN	→ skills.skill_id

Constraints

- `project_skills_project_id_fkey`: `FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE`
- `project_skills_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT`

projects

Projects a user has built.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`project_id`	uuid	PK	NN	gen_random_uuid()
`user_id`	uuid	FK	NN	→ users.user_id
`title`	character varying(120)		NN	
`description`	text		yes	
`repo_url`	web_url		yes	
`started_on`	date		yes	
`completed_on`	date		yes	
`created_at`	timestamp with time zone		NN	now()

Constraints

- `chk_project_dates`: `CHECK (((completed_on IS NULL) OR (started_on IS NULL) OR (completed_on >= started_on)))`
- `projects_description_check`: `CHECK ((length(description) <= 2000))`
- `projects_title_check`: `CHECK ((length(btrim((title)::text)) >= 3))`
- `projects_user_id_fkey`: `FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE`

resumes

Pointer to a parsed resume document in MongoDB.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
resume_id	uuid	PK	NN	gen_random_uuid()
user_id	uuid	FK	NN	→ users.user_id
mongo_doc_id	character(24)	UK	NN	
file_name	character varying(255)		NN	
file_size_bytes	integer		NN	
parse_status	character varying(10)		NN	'pending'::character varying
skills_detected	smallint		NN	0
uploaded_at	timestamp with time zone		NN	now()

Constraints

- resumes_file_size_bytes_check: CHECK (((file_size_bytes >= 1) AND (file_size_bytes <= 5242880)))
- resumes_mongo_doc_id_check: CHECK ((mongo_doc_id ~ '^[0-9a-f]{24}$'::text))
- resumes_parse_status_check: CHECK (((parse_status)::text = ANY ((ARRAY['pending'::character varying, 'parsed'::character varying, 'failed'::character varying])::text[])))
- resumes_skills_detected_check: CHECK ((skills_detected >= 0))
- resumes_user_id_fkey: FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE

roadmap_items

Weak entity: ordered, scored items of a roadmap.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
item_id	uuid	PK	NN	gen_random_uuid()
roadmap_id	uuid	FK, UK	NN	→ roadmaps.roadmap_id
skill_id	integer	FK, UK	NN	→ skills.skill_id
phase	character varying(5)		NN	
sequence_no	smallint	UK	NN	
priority_score	unit_score		NN	
score_breakdown	jsonb		NN	
rationale	text		NN	
status	character varying(5)		NN	'todo'::character varying
completed_at	timestamp with time zone		yes	

Constraints

- chk_done_time: CHECK ((((status)::text = 'done'::text) = (completed_at IS NOT NULL)))
- roadmap_items_phase_check: CHECK (((phase)::text = ANY ((ARRAY['learn'::character varying, 'build'::character varying, 'prove'::character varying])::text[])))
- roadmap_items_score_breakdown_check: CHECK ((jsonb_typeof(score_breakdown) = 'object'::text))
- roadmap_items_sequence_no_check: CHECK ((sequence_no > 0))
- roadmap_items_status_check: CHECK (((status)::text = ANY ((ARRAY['todo'::character varying, 'doing'::character varying, 'done'::character varying])::text[])))
- roadmap_items_roadmap_id_fkey: FOREIGN KEY (roadmap_id) REFERENCES roadmaps(roadmap_id) ON DELETE CASCADE

- `roadmap_items_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT`

roadmaps

Versioned Learn > Build > Prove plan for a goal.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`roadmap_id`	uuid	PK	NN	gen_random_uuid()
`user_id`	uuid	FK	NN	→ users.user_id
`goal_id`	uuid	FK, UK	NN	→ goals.goal_id
`version`	integer	UK	NN	
`status`	character varying(10)		NN	'active'::character varying
`generated_at`	timestamp with time zone		NN	now()

Constraints

- `roadmaps_status_check`: `CHECK (((status)::text = ANY ((ARRAY['active'::character varying, 'archived'::character varying, 'completed'::character varying])::text[])))`
- `roadmaps_version_check`: `CHECK ((version > 0))`
- `roadmaps_goal_id_fkey`: `FOREIGN KEY (goal_id) REFERENCES goals(goal_id) ON DELETE CASCADE`
- `roadmaps_user_id_fkey`: `FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE`

role_skills

M:N: skills a role requires, with level and importance.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`role_id`	integer	FK, PK	NN	→ career_roles.role_id
`skill_id`	integer	FK, PK	NN	→ skills.skill_id
`required_level`	proficiency		NN	
`importance`	numeric(3,2)		NN	

Constraints

- `role_skills_importance_check`: `CHECK (((importance >= 0.05) AND (importance <= (1)::numeric)))`
- `role_skills_required_level_check`: `CHECK (((required_level)::smallint >= 1))`
- `role_skills_role_id_fkey`: `FOREIGN KEY (role_id) REFERENCES career_roles(role_id) ON DELETE CASCADE`
- `role_skills_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT`

scoring_weights

Admin-tunable weights of the 7 priority factors.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`factor`	character varying(20)	PK	NN	
`weight`	numeric(4,3)		NN	
`label`	character varying(80)		NN	

Constraints

- `scoring_weights_factor_check`: `CHECK (((factor)::text = ANY ((ARRAY['demand'::character varying, 'relevance'::character varying, 'gap'::character varying, 'readiness'::character varying, 'interest'::character varying, 'evidence'::character varying, 'ease'::character varying])::text[])))`
- `scoring_weights_weight_check`: `CHECK (((weight >= (0)::numeric) AND (weight <= (1)::numeric)))`

skill_prerequisites

Recursive M:N: skill requires prerequisite skill (kept acyclic by trigger).

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`skill_id`	integer	FK, PK	NN	→ skills.skill_id
`prereq_skill_id`	integer	FK, PK	NN	→ skills.skill_id
`strength`	numeric(3,2)		NN	1.00

Constraints

- `chk_no_self_prereq`: `CHECK ((skill_id <> prereq_skill_id))`
- `skill_prerequisites_strength_check`: `CHECK (((strength > (0)::numeric) AND (strength <= (1)::numeric)))`
- `skill_prerequisites_prereq_skill_id_fkey`: `FOREIGN KEY (prereq_skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE`
- `skill_prerequisites_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE`

skills

Skill catalogue with aliases, effort, FTS vector and embedding.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`skill_id`	integer	PK	NN	nextval('skills_skill_id_seq'::regclass)
`name`	character varying(80)	UK	NN	
`category`	character varying(30)		NN	
`description`	text		NN	
`aliases`	text[]		NN	'{}'::text[]
`effort_hours`	smallint		NN	
`neo4j_node_id`	character varying(64)		yes	
`embedding`	vector(384)		yes	
`search_tsv`	tsvector		yes	GEN (tsvector)
`created_at`	timestamp with time zone		NN	now()

Constraints

- `skills_category_check`: `CHECK (((category)::text = ANY ((ARRAY['language'::character varying, 'frontend'::character varying, 'backend'::character varying, 'data'::character varying, 'ml'::character varying, 'devops'::character varying, 'cloud'::character varying, 'database'::character varying, 'foundation'::character varying, 'soft'::character varying])::text[])))`
- `skills_description_check`: `CHECK ((length(description) >= 10))`

- `skills_effort_hours_check`: `CHECK (((effort_hours >= 1) AND (effort_hours <= 400)))`

user_skills

M:N: skills a user claims; verified level is derived from evidence.

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`user_id`	uuid	FK, PK	NN	→ users.user_id
`skill_id`	integer	FK, PK	NN	→ skills.skill_id
`claimed_level`	proficiency		NN	
`verified_level`	proficiency		NN	0
`source`	character varying(10)		NN	'self'::character varying
`updated_at`	timestamp with time zone		NN	now()

Constraints

- `user_skills_claimed_level_check`: `CHECK (((claimed_level)::smallint >= 1))`
- `user_skills_source_check`: `CHECK (((source)::text = ANY ((ARRAY['self'::character varying, 'resume'::character varying, 'evidence'::character varying])::text[])))`
- `user_skills_skill_id_fkey`: `FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE RESTRICT`
- `user_skills_user_id_fkey`: `FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE`

users

Login identity and role (RBAC).

COLUMN	TYPE	KEYS	NULL	DEFAULT / NOTES
`user_id`	uuid	PK	NN	gen_random_uuid()
`email`	email_address	UK	NN	
`password_hash`	text		NN	
`role`	user_role		NN	'student'::user_role
`is_active`	boolean		NN	true
`last_login_at`	timestamp with time zone		yes	
`created_at`	timestamp with time zone		NN	now()
`updated_at`	timestamp with time zone		NN	now()

Constraints

- `users_password_hash_check`: `CHECK ((password_hash ~ '^\$2[aby]\$\d{2}\$.{53}$'::text))`